Actividad 2.1: alarmas de CloudWatch, avisos y recuperación automática

Administración de Recursos de Computación en la Nube - UD02: Monitorización y alta disponibilidad en máquinas virtuales

Alumno: Pau López Núñez (iniciales pln)
Fecha: 12/11/2026
Entorno: AWS Academy Learner Lab, región us-east-1

Introducción

En esta actividad he vigilado una instancia EC2 con Amazon CloudWatch: he consultado sus métricas desde la consola y desde la AWS CLI, he creado alarmas que avisan por correo con un tema de SNS y alarmas que detienen, recuperan o reinician la instancia, he añadido una comprobación de estado de la aplicación y lo he reunido todo en un panel. En todos los nombres he sustituido xx por mis iniciales, pln.

La infraestructura la he creado desde la consola y la CLI la he usado desde el terminal de mi equipo, con las credenciales del laboratorio copiadas en ~/.aws/credentials. Los comandos de la CLI están en la carpeta scripts del repositorio y su salida completa en evidencias.

Recursos creados: instancia mon-pln (i-07df4a781ae3b354d), grupo de seguridad mon-pln-sg, tema SNS avisos-pln, alarmas mon-pln-sistema, mon-pln-cpu y mon-pln-instancia, comprobación de aplicación apache-pln, panel panel-pln y, en la ampliación, la alarma mon-pln-cpu-cli.

Parte 1: Preparación

He lanzado la instancia desde la consola de EC2 con estos datos:

- Nombre mon-pln y etiquetas Proyecto = UD02 y Propietario = pln, aplicadas a instancias y volúmenes.
- AMI Ubuntu Server 26.04 LTS (x86) y tipo t3.micro, con el par de claves vockey.
- Grupo de seguridad nuevo mon-pln-sg con SSH y HTTP desde 213.0.86.0/23, el rango del instituto (mi IP era 213.0.87.110).
- Perfil de instancia LabInstanceProfile y el script de datos de usuario que instala stress-ng y apache2.

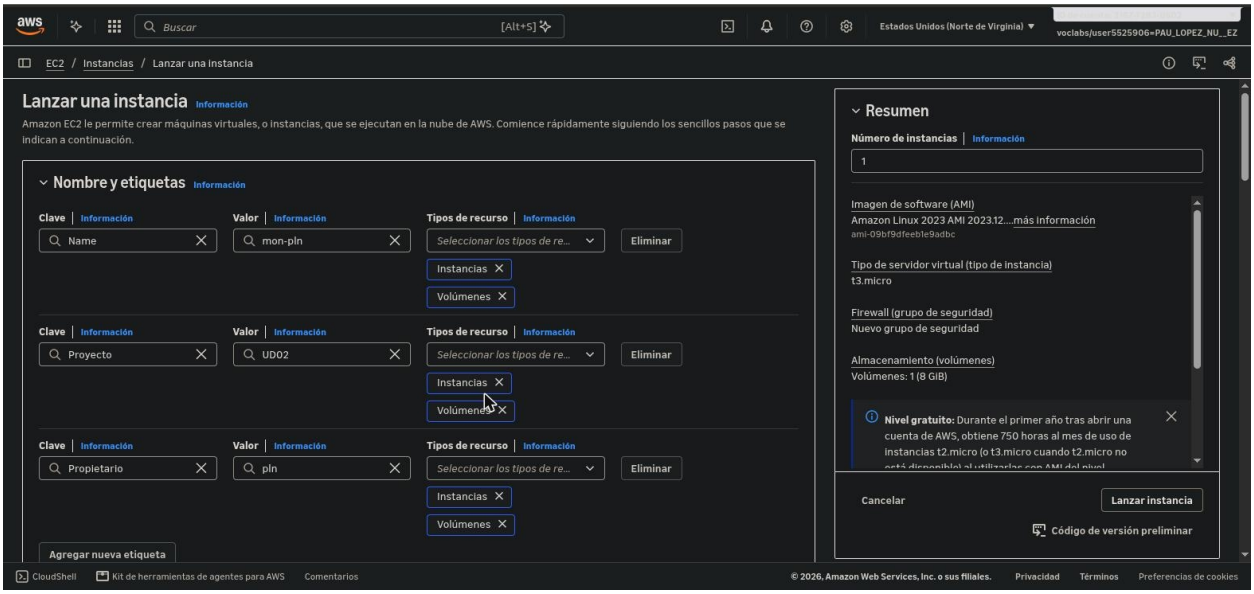


Figura 1. Nombre y etiquetas aplicadas a instancias y volúmenes

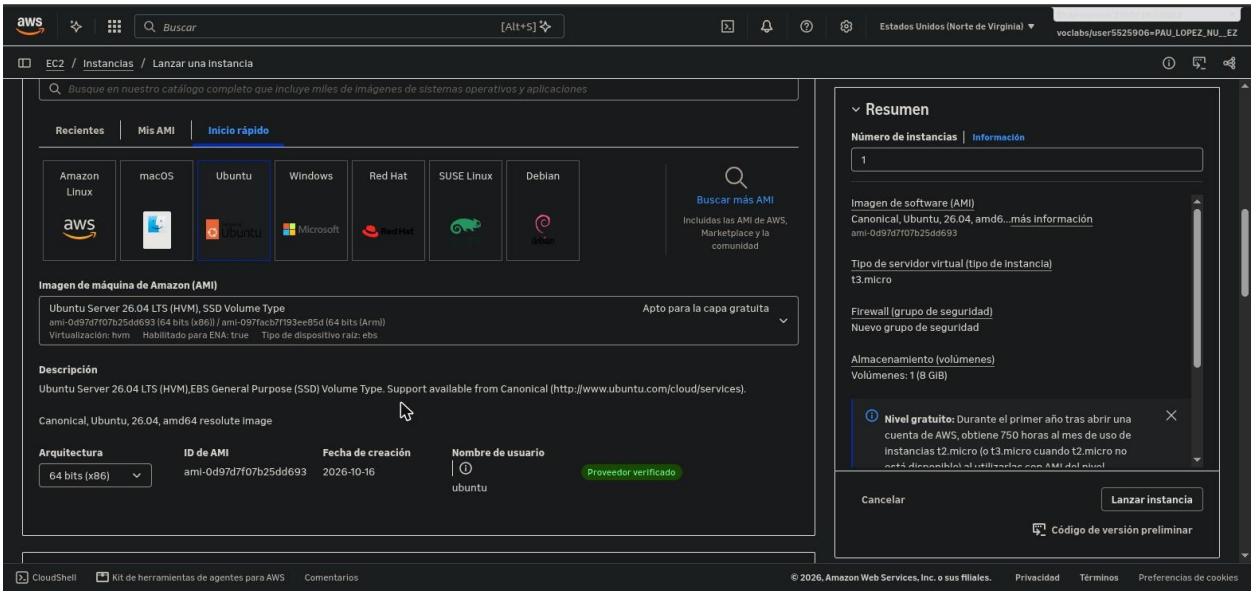


Figura 2. AMI Ubuntu Server 26.04 LTS de 64 bits (x86)

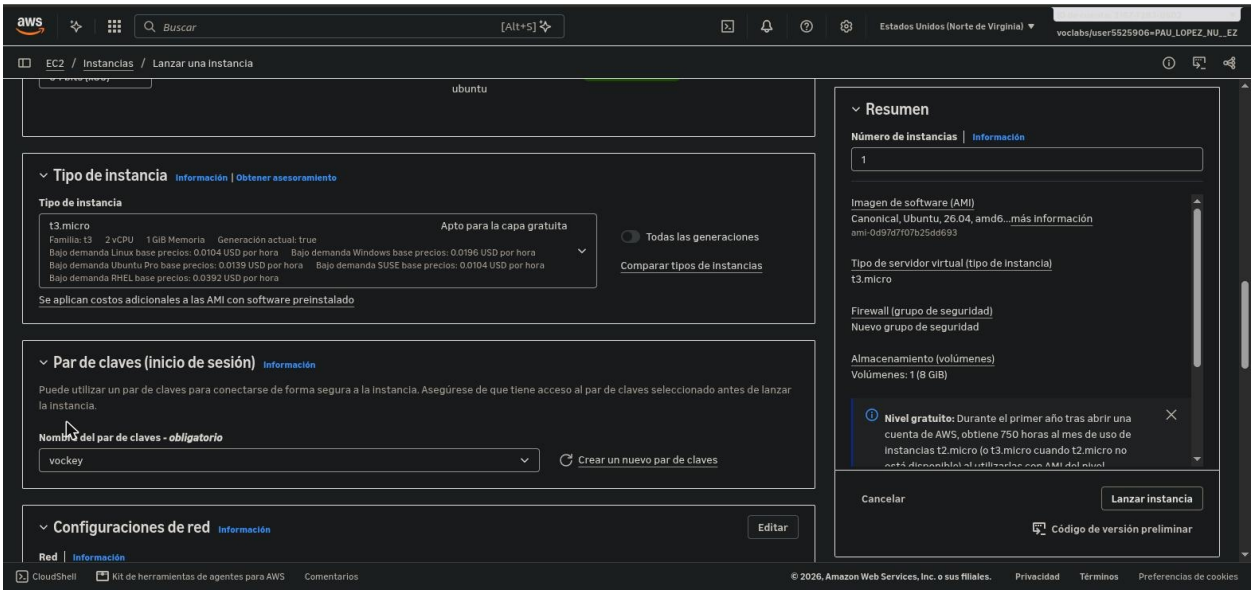


Figura 3. Tipo t3.micro y par de claves vockey

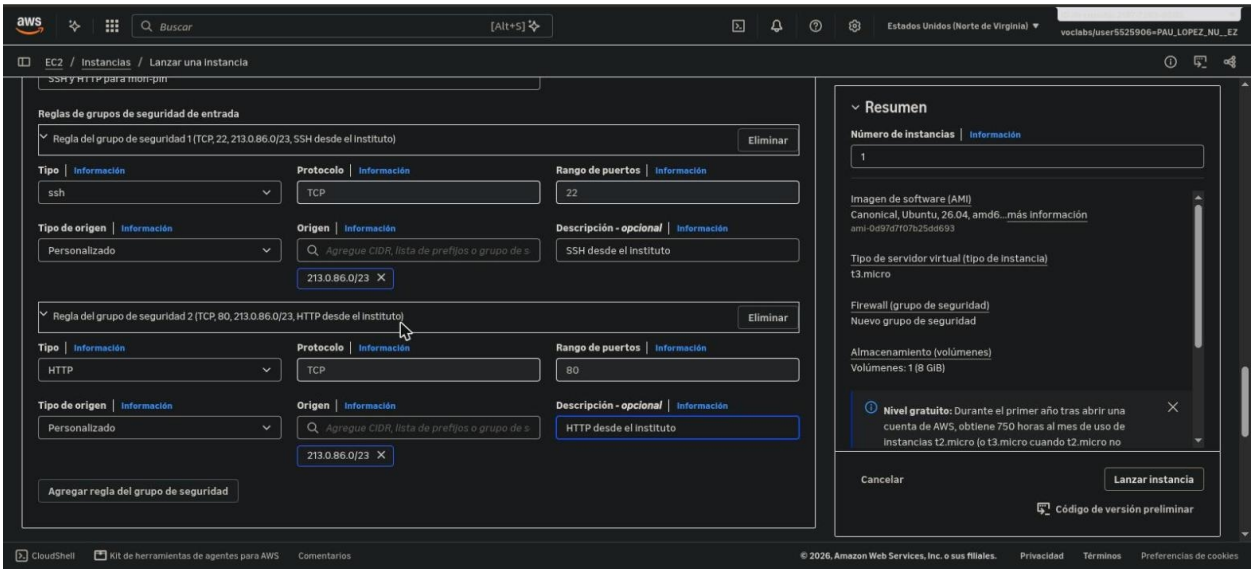


Figura 4. Grupo de seguridad mon-pln-sg con SSH y HTTP desde 213.0.86.0/23

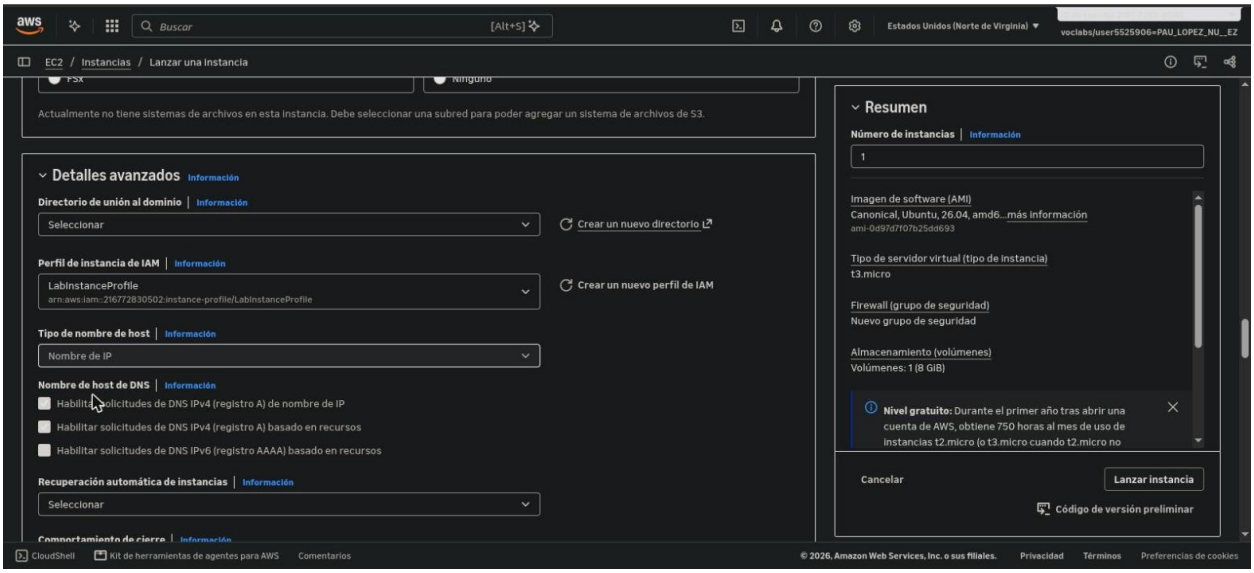


Figura 5. Perfil de instancia LabInstanceProfile

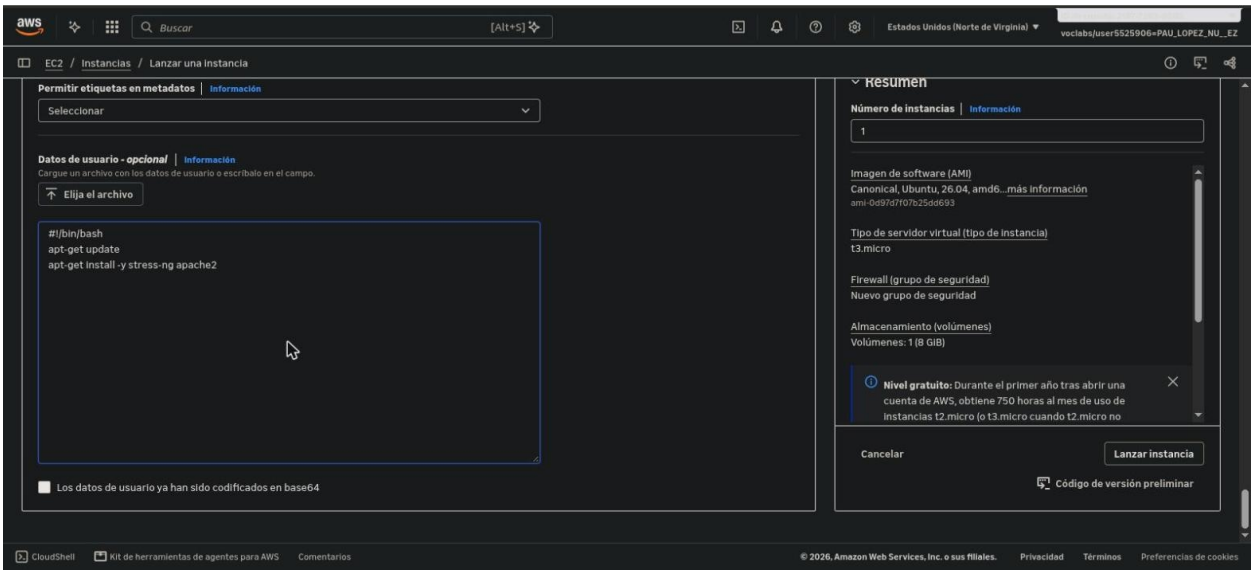


Figura 6. Datos de usuario

```
#!/bin/bash
apt-get update
apt-get install -y stress-ng apache2
```

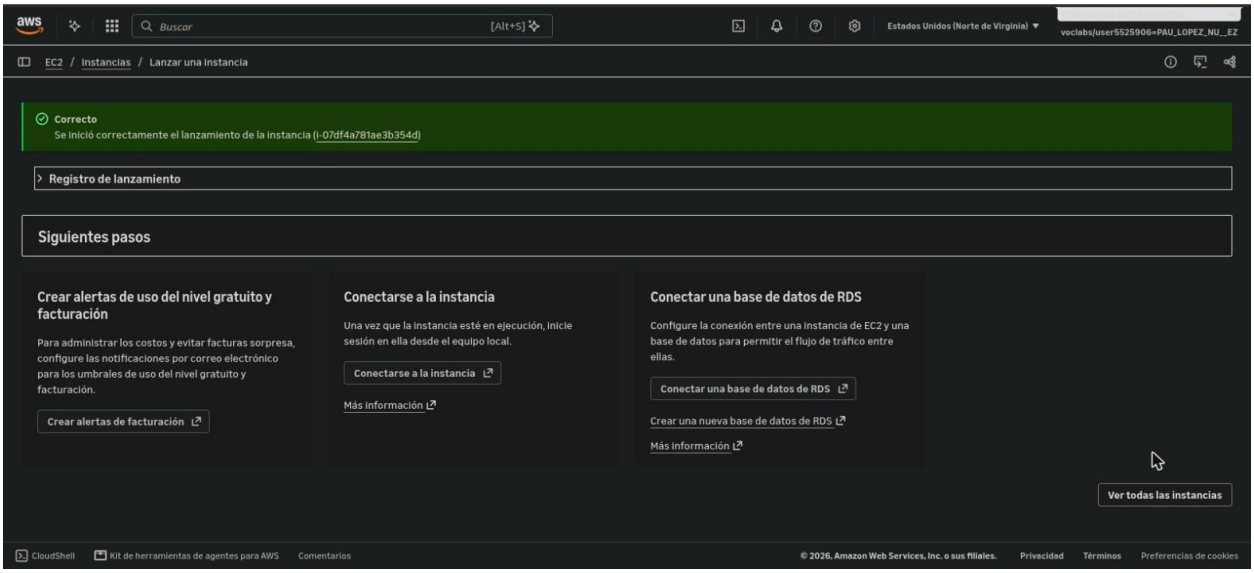


Figura 7. Instancia lanzada (i-07df4a781ae3b354d)

Desde el terminal he guardado el ID de la instancia en una variable, como pide el enunciado, y he comprobado que Apache responde:

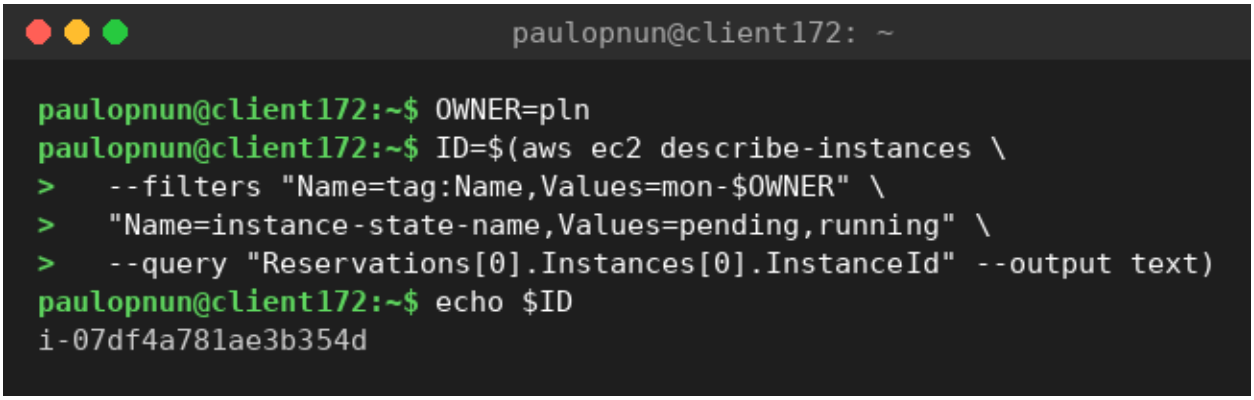


Figura 8. Variable ID desde la CLI y respuesta 200 OK de Apache



Figura 9. Página por defecto de Apache en http://3.90.113.250

Parte 2: Métricas de EC2

Con list-metrics he visto qué métricas publica EC2 para la instancia sin instalar nada. Entre ellas están CPUUtilization, el tráfico de red, los créditos de CPU y las comprobaciones de estado, pero no la memoria ni el disco.

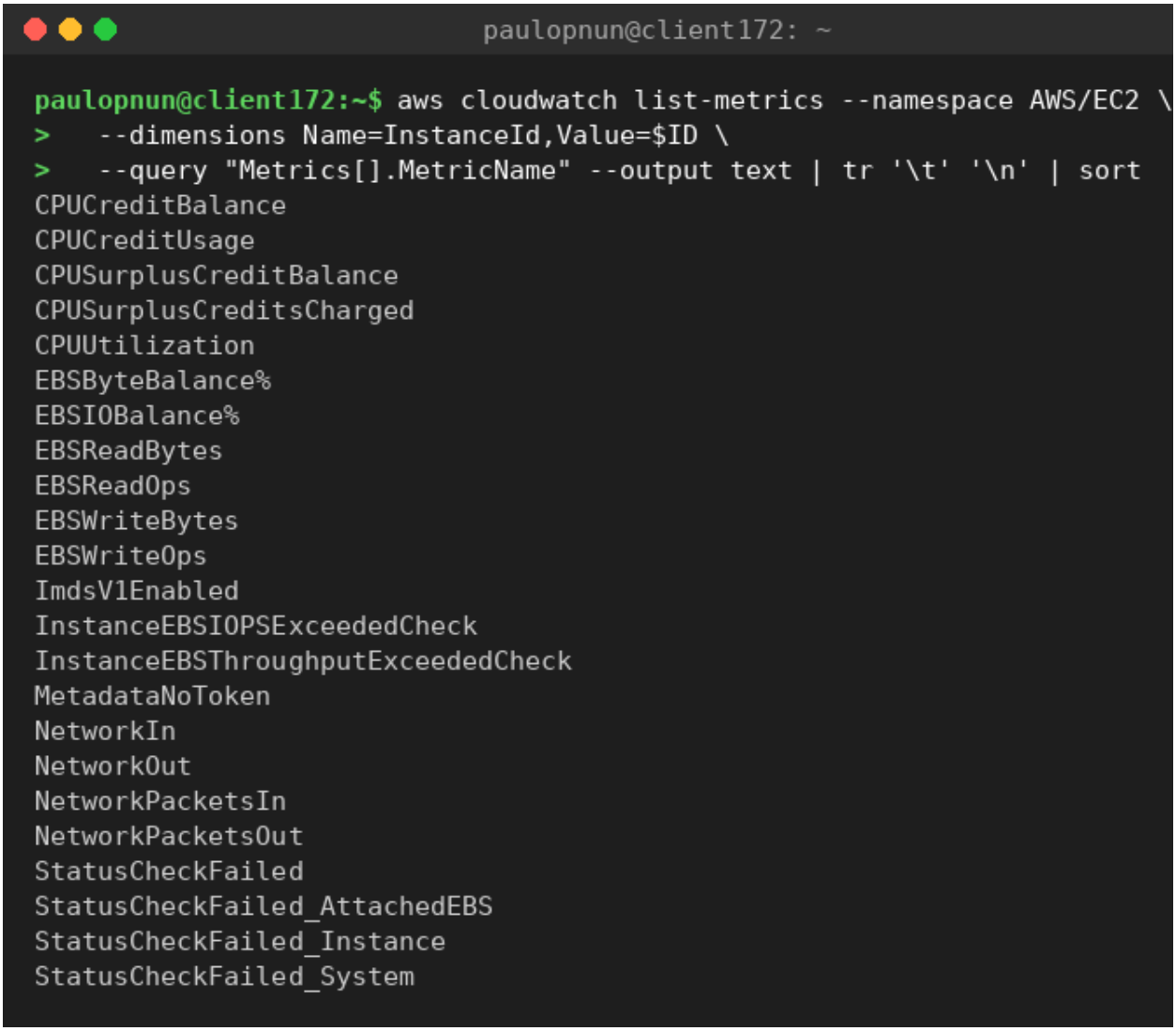


Figura 10. Métricas disponibles en AWS/EC2 para mon-pln

Después he consultado la CPU con periodo de 300 y de 60 segundos, y StatusCheckFailed_System con Máximo y 60 segundos:

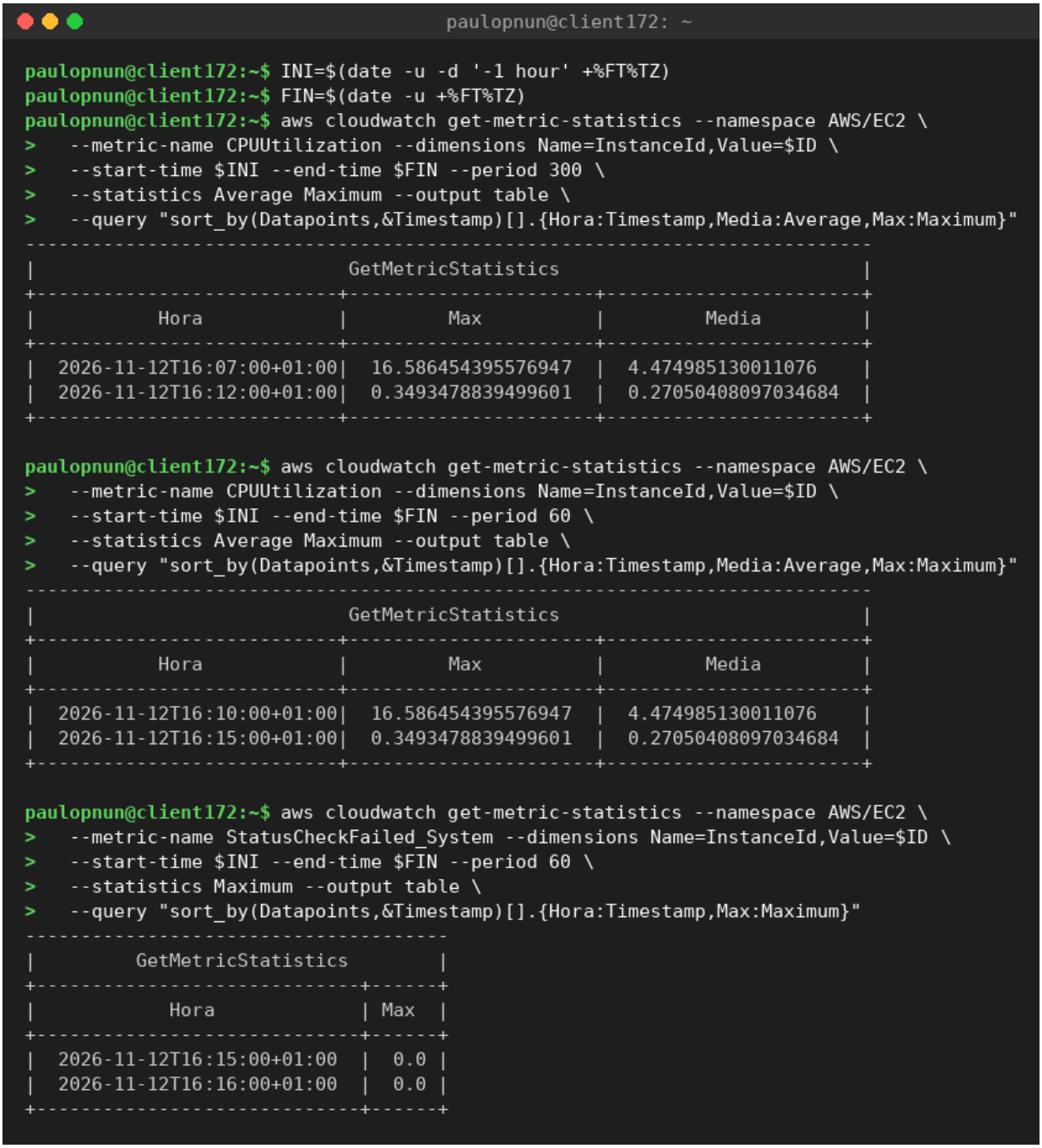


Figura 11. CPU con monitorización básica y comprobación del sistema

- Con --period 60 no salen más datos: siguen siendo dos, uno cada 5 minutos, porque la monitorización básica solo envía un dato cada 5 minutos.
- StatusCheckFailed_System sí tiene un dato por minuto, todos con valor 0.0: las comprobaciones se publican cada minuto aunque la monitorización sea básica, y el host está sano.

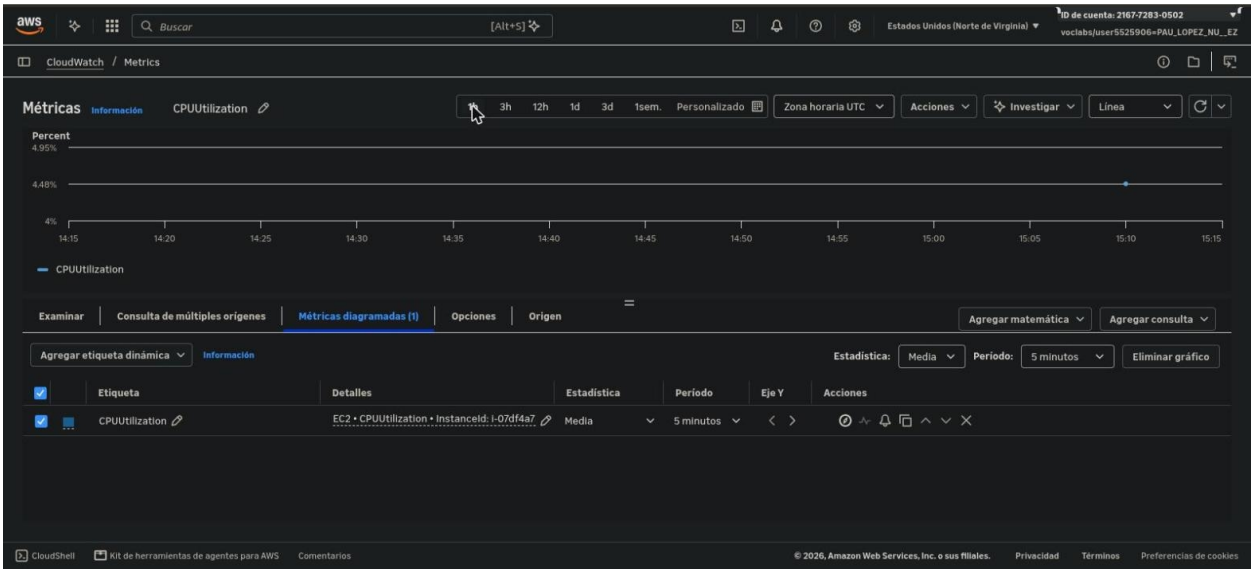


Figura 12. CPUUtilization en CloudWatch > Métricas

He activado la monitorización detallada con aws ec2 monitor-instances. Unos diez minutos después, la misma consulta con --period 60 ya devuelve un dato por minuto desde las 16:19:

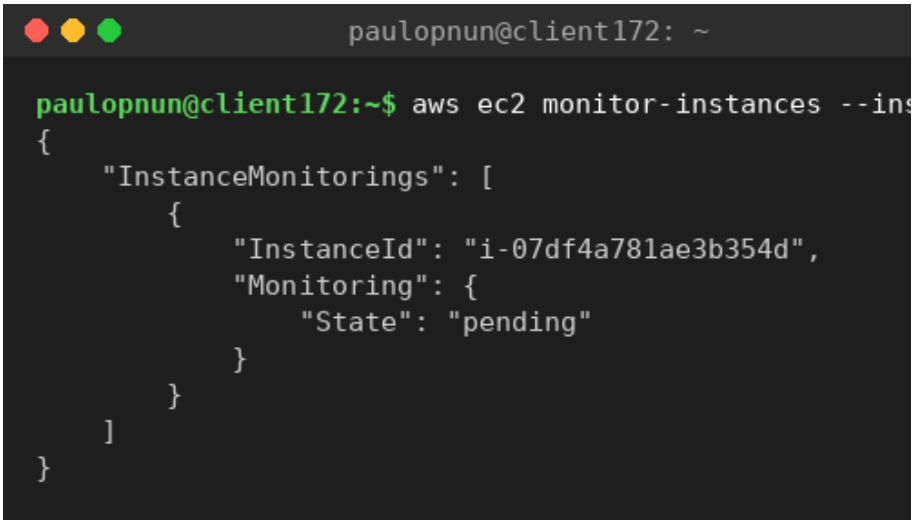


Figura 13. Activación de la monitorización detallada

```
paulopnun@client172: ~

paulopnun@client172:~$ aws ec2 describe-instances --instance-ids $ID \
>   --query "Reservations[0].Instances[0].Monitoring.State" --output text
enabled

paulopnun@client172:~$ INI=$(date -u -d '-15 minutes' +%FT%TZ)
paulopnun@client172:~$ FIN=$(date -u +%FT%TZ)
paulopnun@client172:~$ aws cloudwatch get-metric-statistics --namespace AWS/EC2 \
>   --metric-name CPUUtilization --dimensions Name=InstanceId,Value=$ID \
>   --start-time $INI --end-time $FIN --period 60 \
>   --statistics Average Maximum --output table \
>   --query "sort_by(Datapoints,&Timestamp)[].{Hora:Timestamp,Media:Average,Max:Maximum}"
-------------------------------------------------------------------------
|                        GetMetricStatistics                            |
+--------------------------+----------------------+----------------------+
|          Hora            |         Max          |        Media         |
+--------------------------+----------------------+----------------------+
|  2026-11-12T16:15:00+01:00|  0.3493478839499601  |  0.23318017469344757 |
|  2026-11-12T16:19:00+01:00|  0.1583306944884251  |  0.1583306944884251  |
|  2026-11-12T16:20:00+01:00|  0.21708636697615388 |  0.21708636697615388 |
|  2026-11-12T16:21:00+01:00|  0.19999999999999998 |  0.19999999999999998 |
|  2026-11-12T16:22:00+01:00|  0.1998501124156882  |  0.1998501124156882  |
|  2026-11-12T16:23:00+01:00|  0.1832569762598917  |  0.1832569762598917  |
|  2026-11-12T16:24:00+01:00|  0.1915134559019451  |  0.1915134559019451  |
|  2026-11-12T16:25:00+01:00|  0.2333294445092582  |  0.2333294445092582  |
+--------------------------+----------------------+----------------------+
```

Figura 14. CPU con monitorización detallada: un dato por minuto

Para pensar: la monitorización detallada tiene coste. ¿Cuándo compensa?

Una alarma que necesita tres datos seguidos por encima del umbral tarda como mínimo 15 minutos en saltar si llega un dato cada 5 minutos, y en la práctica algo más por el retraso con el que llegan. Con la detallada son unos 3 minutos. Compensa en instancias de producción en las que reaccionar 10 minutos antes importa (alarmas de CPU, escalado automático, recuperación) y no compensa en máquinas de pruebas o en las que basta con ver tendencias.

Para pensar: ¿por qué EC2 no puede medir la memoria ni el espacio libre en disco?

Porque EC2 mide desde fuera, desde el hipervisor: ve cuánta CPU usa la máquina virtual, cuánto tráfico pasa por su interfaz y cuántas operaciones hace contra el volumen, pero no sabe cómo usa el sistema operativo la memoria que tiene asignada ni qué ocupan los ficheros dentro del sistema de ficheros. Para eso hay que instalar dentro de la instancia el agente de CloudWatch, que publica esas métricas en el espacio de nombres CWAgent.

Parte 3: Recuperación ante un fallo del sistema

Primero he comprobado que la recuperación automática simplificada está en Predeterminada, es decir, activada:

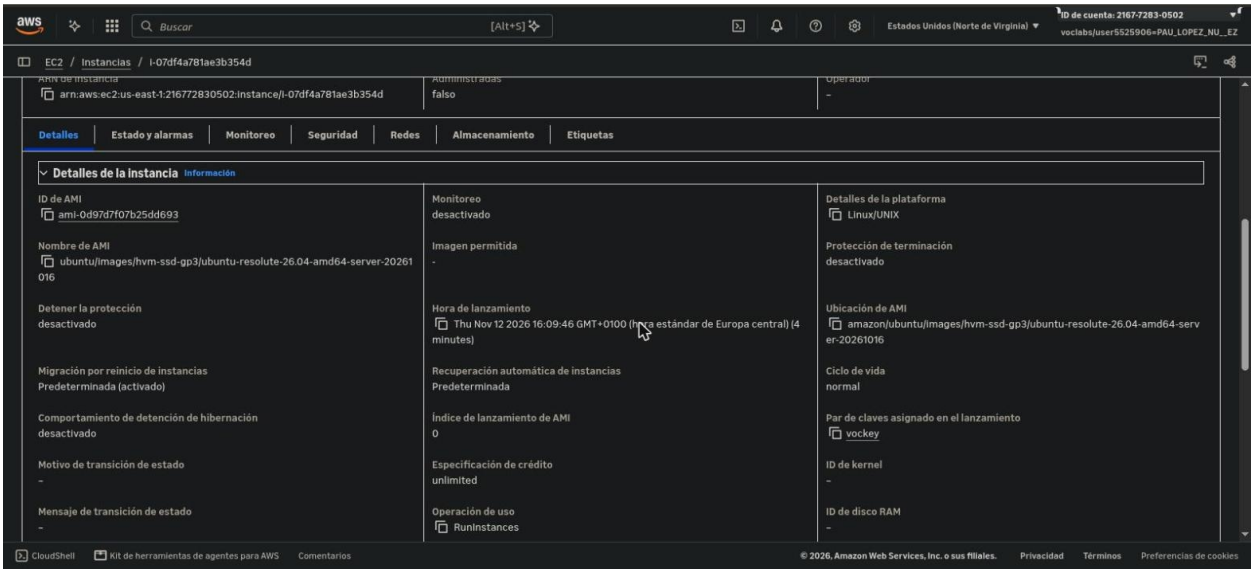


Figura 15. Recuperación automática de instancias: Predeterminada

Desde Acciones > Supervisar y solucionar problemas > Administrar alarmas de CloudWatch he creado mon-pln-sistema: sin notificación, acción Detener, Máximo de la comprobación del sistema fallida, 2 periodos consecutivos de 1 minuto.

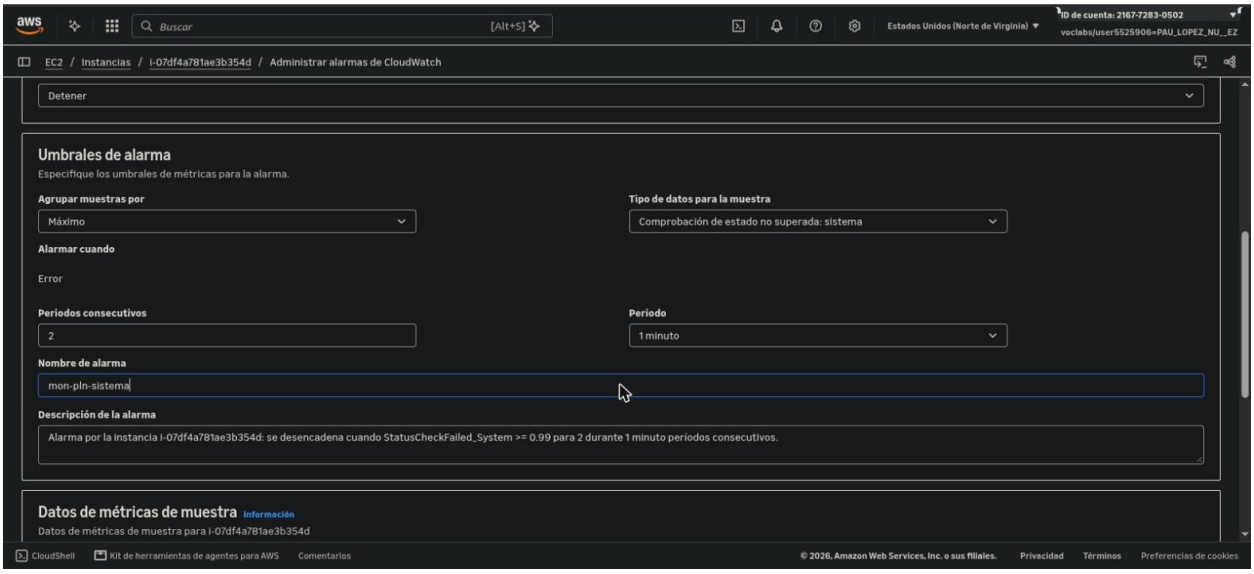


Figura 16. Configuración de mon-pln-sistema

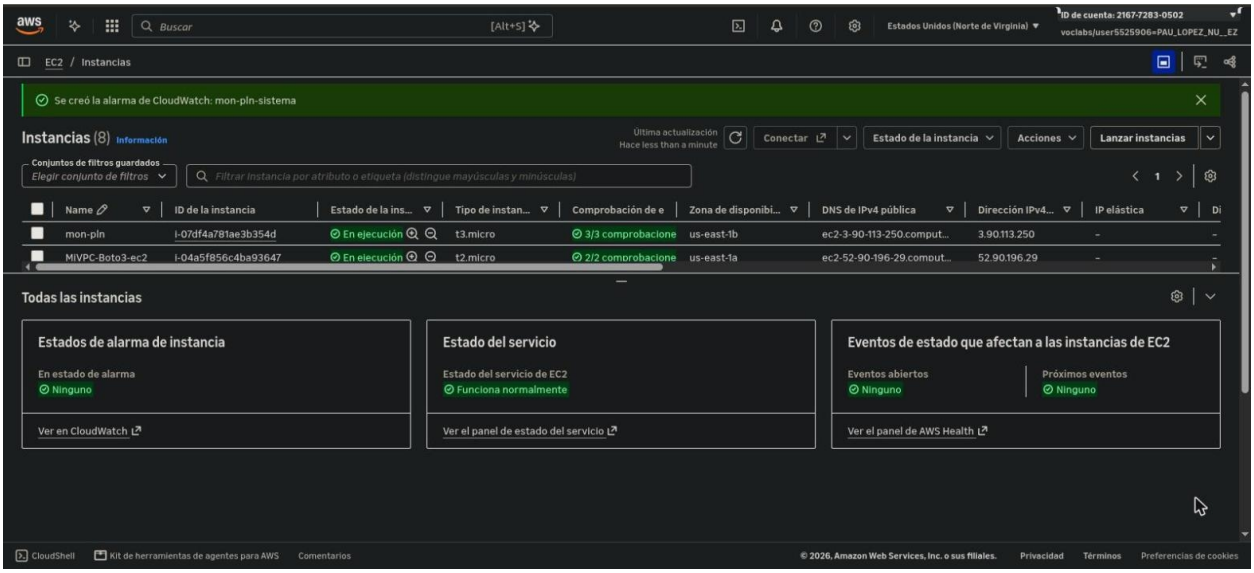


Figura 17. Alarma creada

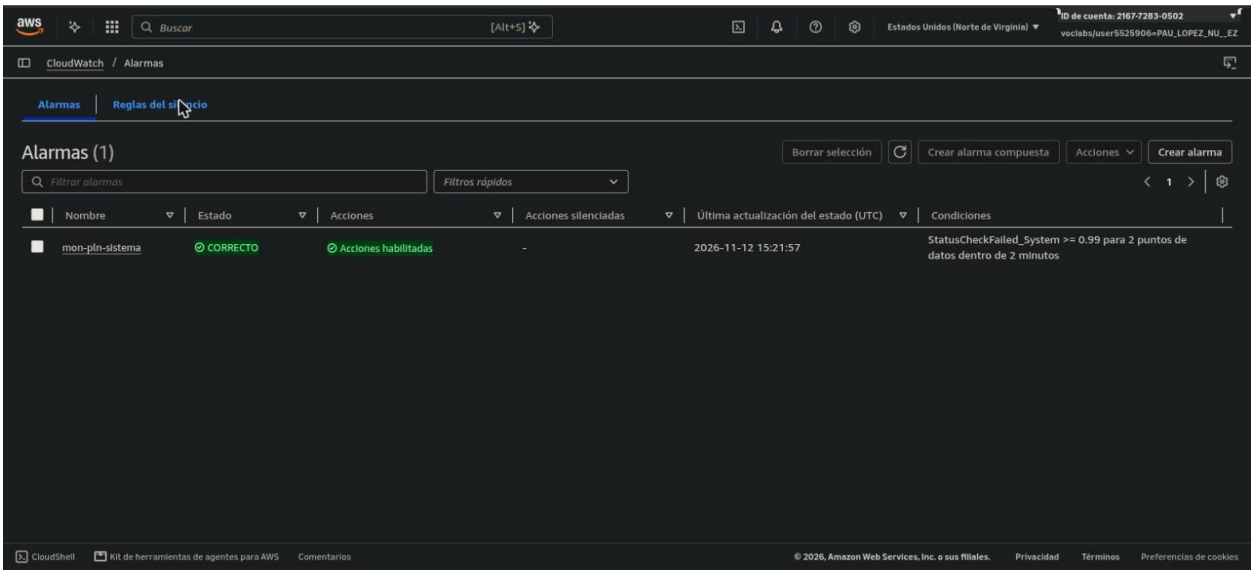


Figura 18. A los pocos minutos pasa de Datos insuficientes a OK

Para simular el fallo del host he forzado el estado de la alarma con set-alarm-state. La alarma pasa a En alarma y la instancia empieza a detenerse en segundos:

```
paulopnun@client172: ~

paulopnun@client172:~$ aws cloudwatch set-alarm-state --alarm-name mon-$OWNER-sistema \
>   --state-value ALARM --state-reason "Simulacro de fallo del host"

paulopnun@client172:~$ aws ec2 describe-instances --instance-ids $ID \
>   --query "Reservations[0].Instances[0].State.Name" --output text
stopping

paulopnun@client172:~$ aws cloudwatch describe-alarm-history --alarm-name mon-$OWNER-sistema \
>   --output table --query "AlarmHistoryItems[].{Hora:Timestamp,Tipo:HistoryItemType,Resumen:HistorySummary}"
-------------------------------------------------------------------------------------------------------------------
|                                                DescribeAlarmHistory                                             |
+-----------------------------------+-------------------------------------------------------------+---------------------+
|              Hora                 |                           Resumen                           |        Tipo         |
+-----------------------------------+-------------------------------------------------------------+---------------------+
|  2026-11-12T16:27:00.251000+01:00 |  Stop EC2 Instance 'i-07df4a781ae3b354d' action completed successfully  |  Action             |
|  2026-11-12T16:26:59.851000+01:00 |  Alarm updated from OK to ALARM                             |  StateUpdate        |
|  2026-11-12T16:21:57.086000+01:00 |  Alarm updated from INSUFFICIENT_DATA to OK                  |  StateUpdate        |
|  2026-11-12T16:20:54.268000+01:00 |  Alarm "mon-pln-sistema" created                            |  ConfigurationUpdate |
+-----------------------------------+-------------------------------------------------------------+---------------------+
```

Figura 19. set-alarm-state, instancia en stopping e historial con la acción ejecutada

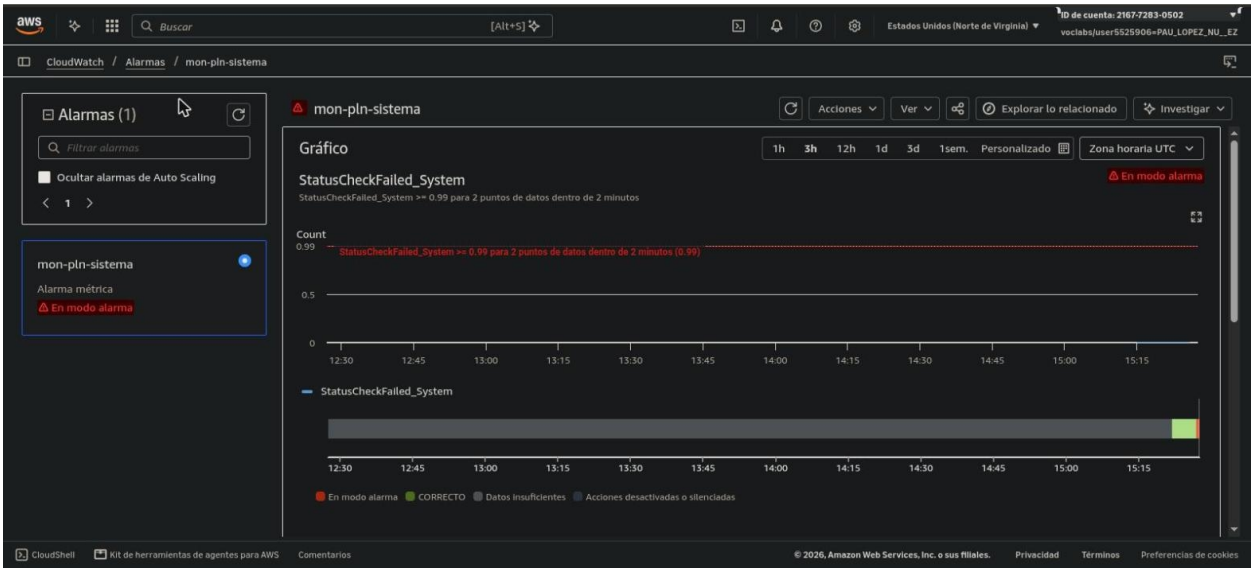


Figura 20. mon-pln-sistema En alarma

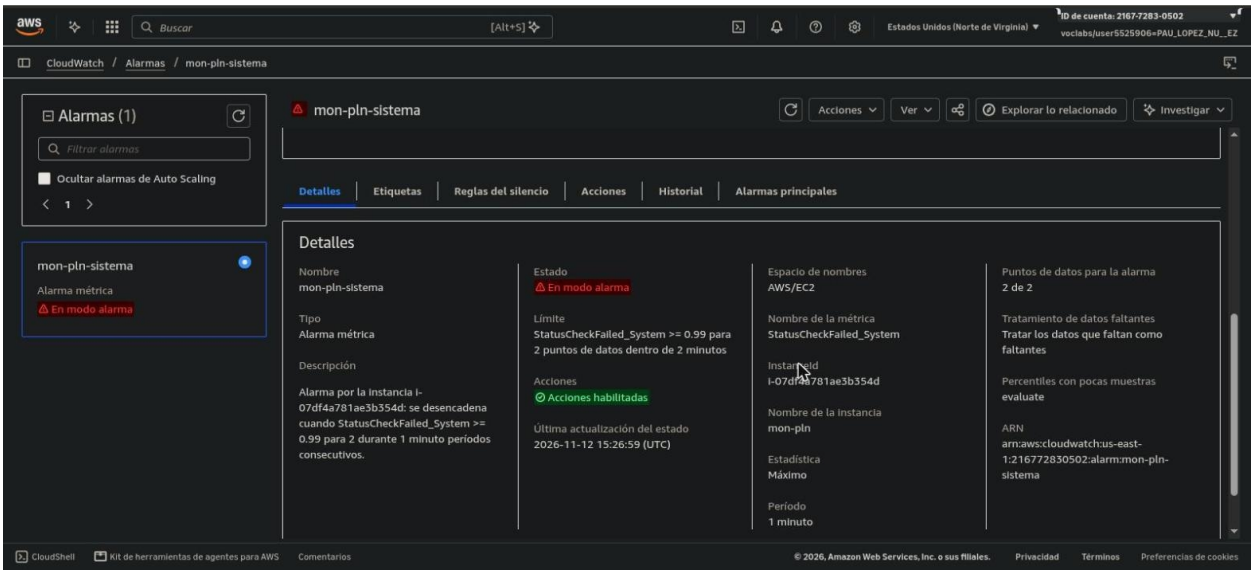


Figura 21. Detalles de la alarma: 2 de 2, Máximo, 1 minuto

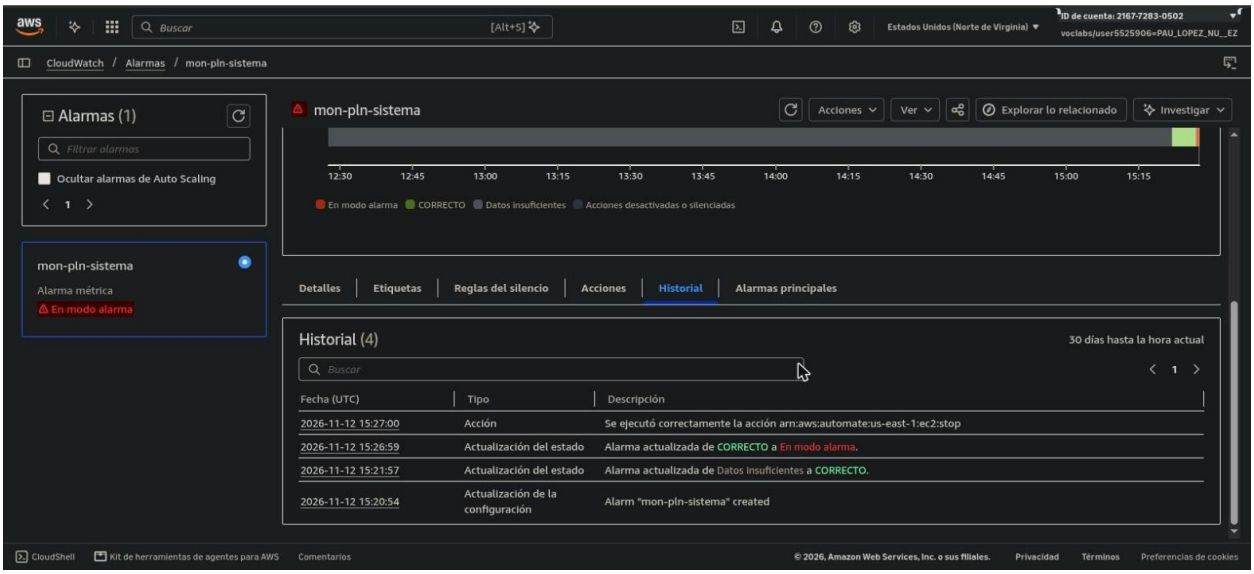


Figura 22. Pestaña Historial: cambio a En alarma y acción arn:aws:automate:us-east-1:ec2:stop

```
paulopnun@client172: ~
paulopnun@client172:~$ aws cloudwatch describe-alarm-history --alarm-name mon-$OWNER-sistema \
>   --history-item-type StateUpdate --output table \
>   --query "AlarmHistoryItems[].{Hora:Timestamp,Resumen:HistorySummary}"
-------------------------------------------------------------------------------
|                          DescribeAlarmHistory                               |
+-----------------------------------+-----------------------------------------+
|              Hora                 |                 Resumen                 |
+-----------------------------------+-----------------------------------------+
|  2026-11-12T16:35:14.006000+01:00 |  Alarm updated from OK to INSUFFICIENT_DATA  |
|  2026-11-12T16:28:14.004000+01:00 |  Alarm updated from ALARM to OK         |
|  2026-11-12T16:26:59.851000+01:00 |  Alarm updated from OK to ALARM         |
|  2026-11-12T16:21:57.086000+01:00 |  Alarm updated from INSUFFICIENT_DATA to OK  |
+-----------------------------------+-----------------------------------------+

paulopnun@client172:~$ aws ec2 start-instances --instance-ids $ID --output table \
>   --query "StartingInstances[].{ID:InstanceId,Antes:PreviousState.Name,Ahora:CurrentState.Name}"
-----------------------------------------------
|              StartInstances                 |
+---------+----------+------------------------+
|  Ahora  |  Antes   |          ID            |
+---------+----------+------------------------+
|  pending|  stopped |  i-07df4a781ae3b354d   |
+---------+----------+------------------------+
```

Figura 23. Evolución de la alarma con la instancia detenida y nuevo arranque

Para pensar: ¿en qué estado estará la alarma unos minutos después, con la instancia detenida? ¿Por qué no vuelve a OK?

En mi caso volvió a OK en la siguiente evaluación (16:28), porque los datos del minuto anterior a la parada todavía eran 0, y a las 16:35 pasó a Datos insuficientes. set-alarm-state solo cambia el estado hasta la siguiente evaluación; después la alarma se evalúa con los datos reales. Con la instancia detenida dejan de publicarse

comprobaciones de estado, así que se queda sin datos y, con el tratamiento por defecto (ausentes), acaba en Datos insuficientes y no en OK.

Parte 4: Avisos por correo con SNS

He creado el tema estándar avisos-pln y una suscripción por correo electrónico a mi dirección, que he confirmado desde el correo de AWS Notifications:

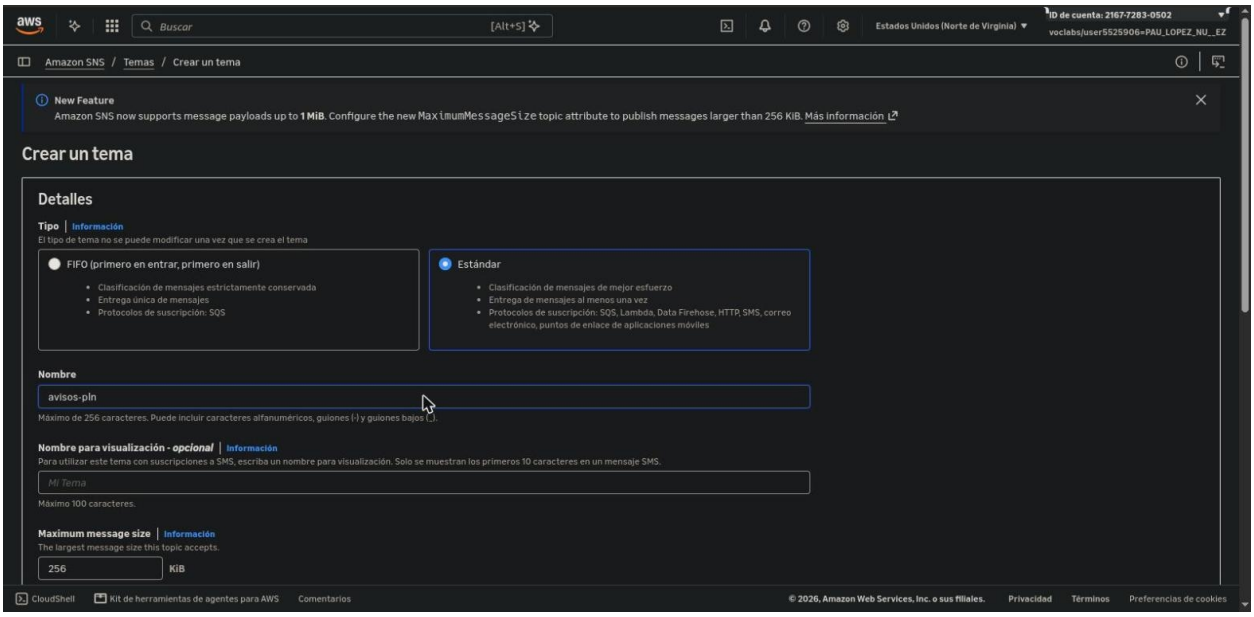


Figura 24. Creación del tema estándar avisos-pln

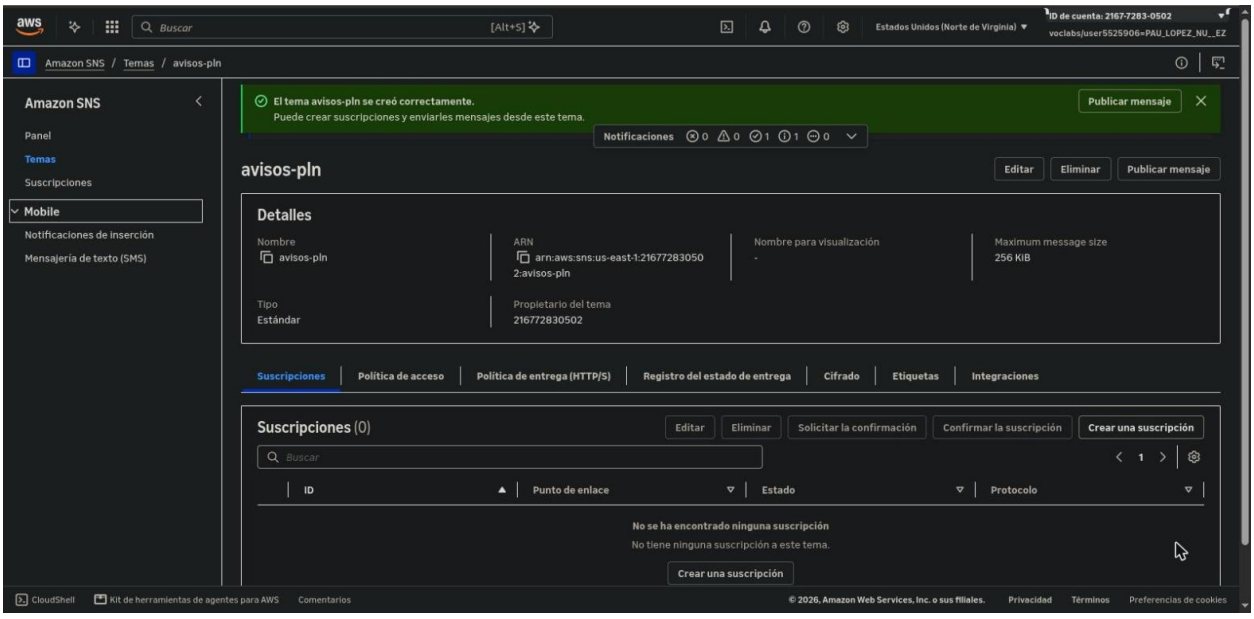


Figura 25. Tema creado

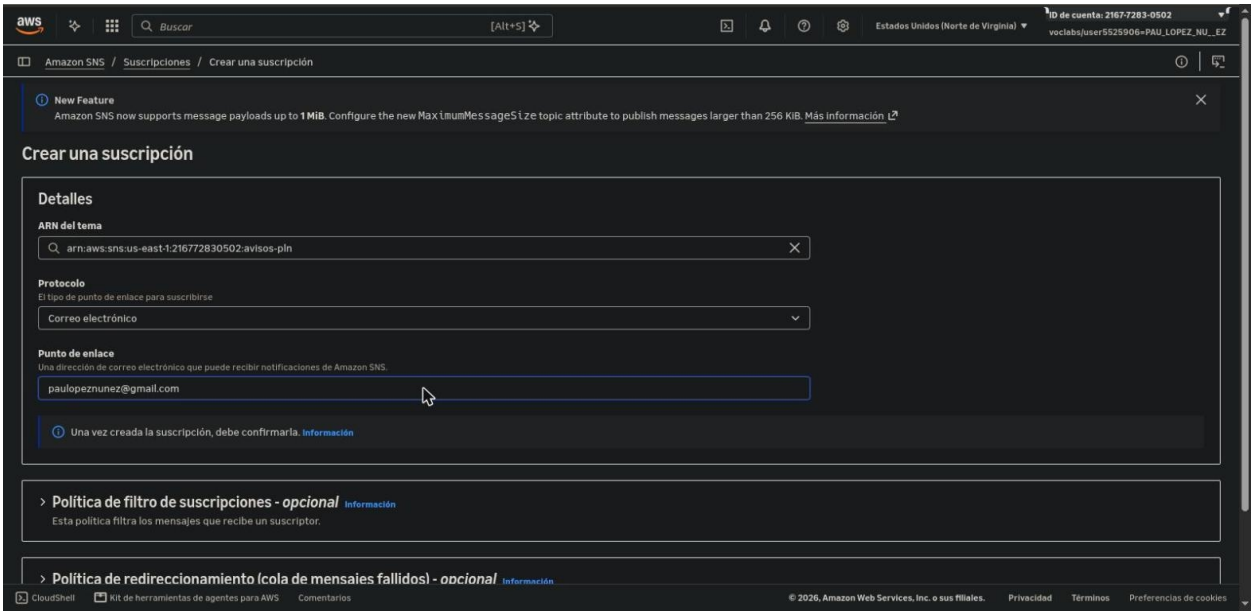


Figura 26. Suscripción por correo electrónico

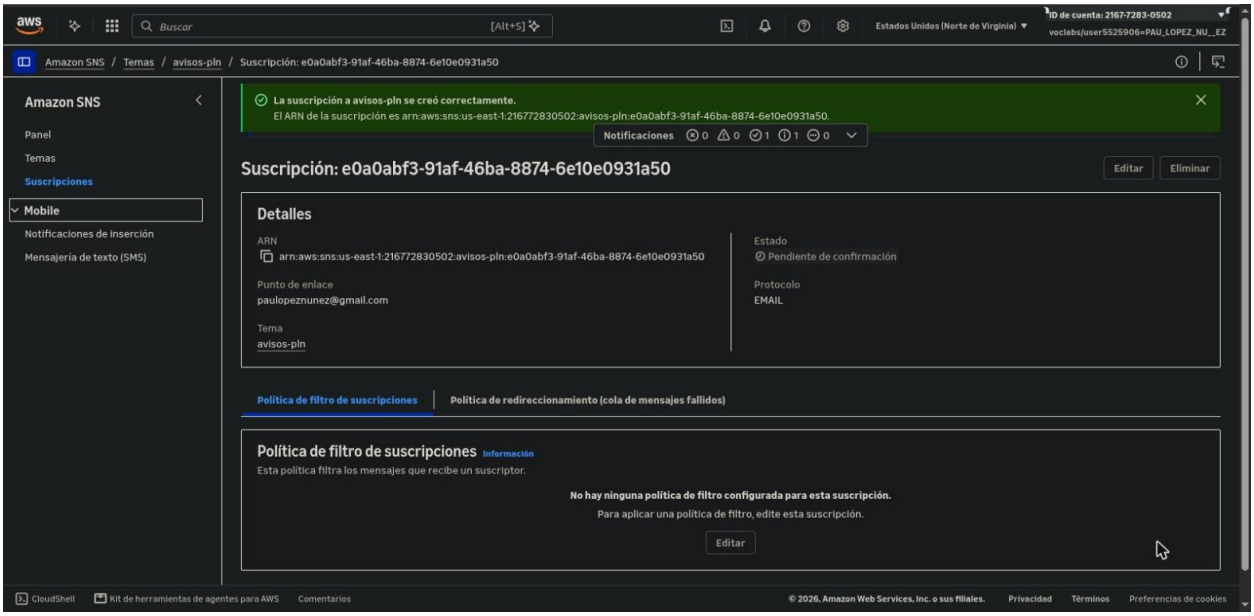


Figura 27. Suscripción pendiente de confirmación

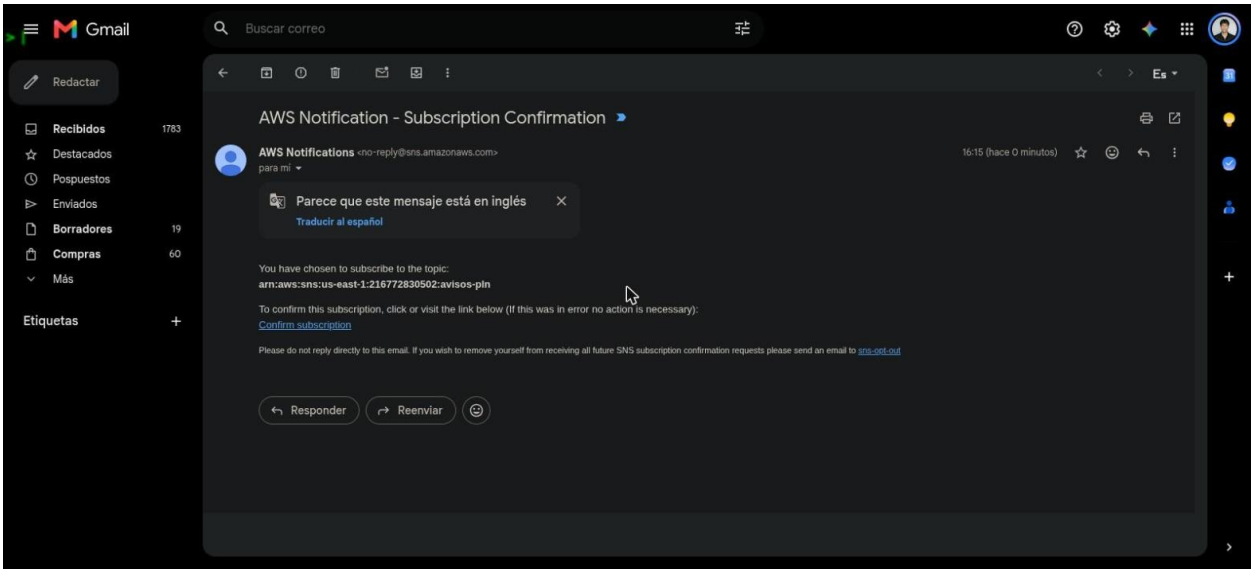


Figura 28. Correo de confirmación

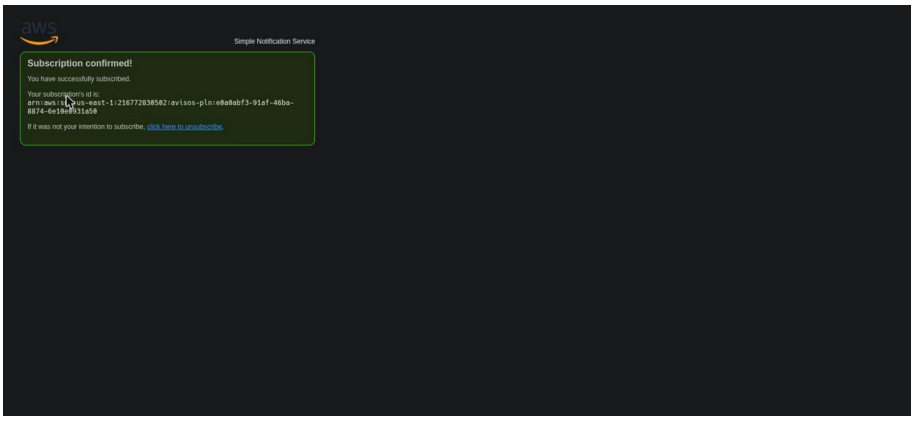


Figura 29. Suscripción confirmada

Con el ARN del tema he enviado un mensaje de prueba con aws sns publish, que llegó al correo:



Figura 30. ARN del tema, suscripción y mensaje de prueba

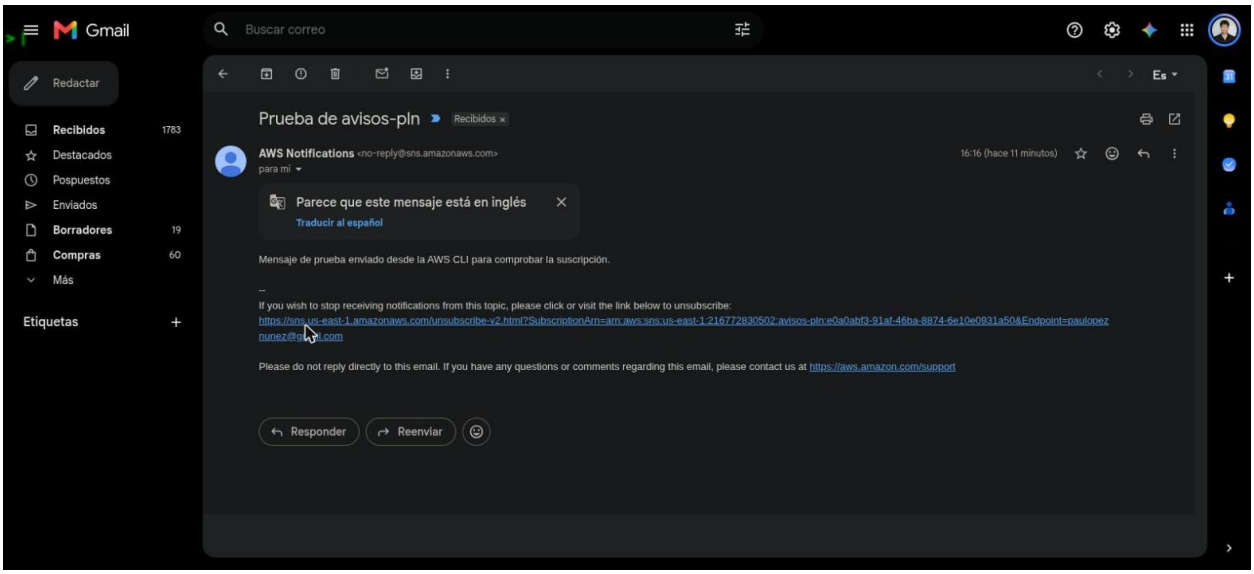


Figura 31. Mensaje de prueba recibido

Después he dejado mon-pln-sistema como estaría en producción, editándola en CloudWatch: notificación a avisos-pln en alarma y acción Recuperar esta instancia en vez de Detener.

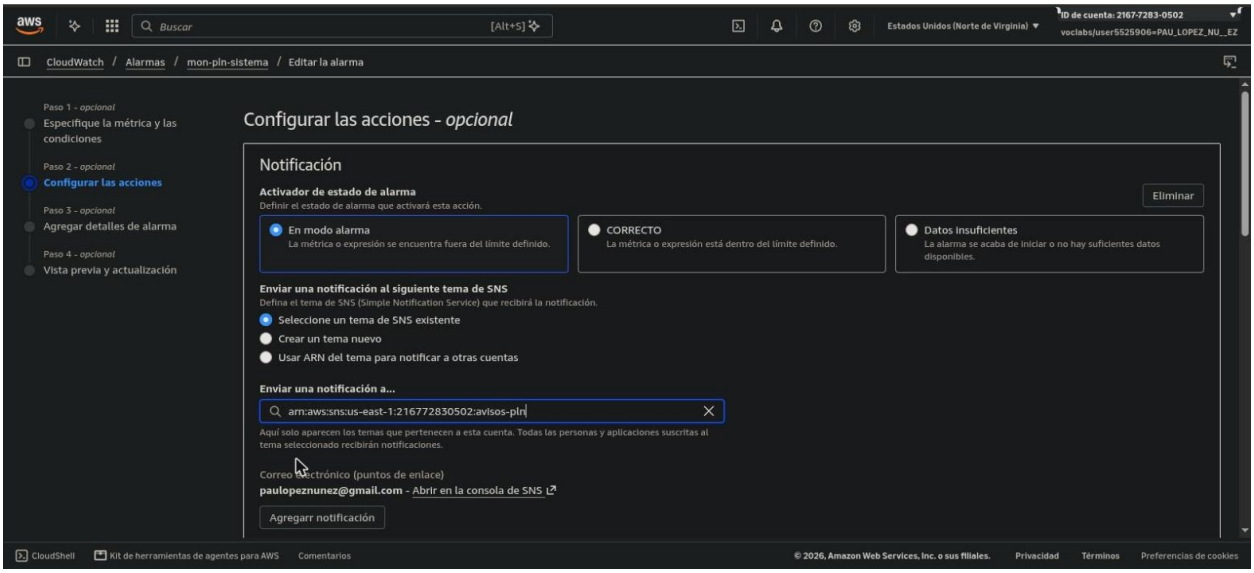


Figura 32. Notificación a avisos-pln

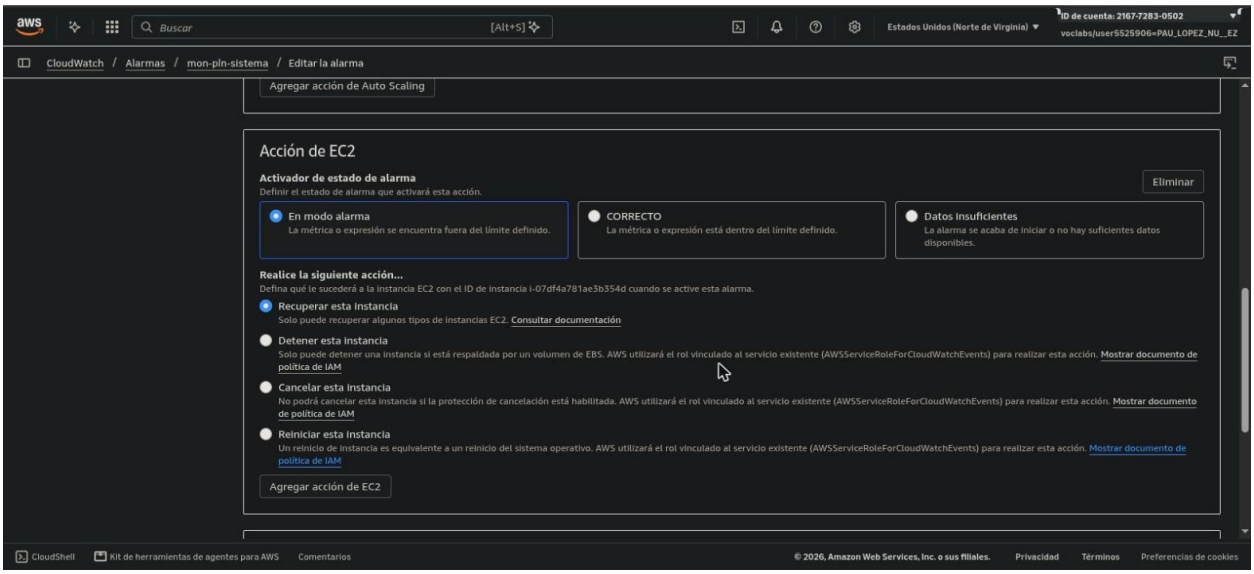


Figura 33. Acción Recuperar esta instancia

```
paulopnun@client172: ~
paulopnun@client172:~$ aws cloudwatch describe-alarms --alarm-names m
>   --query "MetricAlarms[0].AlarmActions" --output table
-------------------------------------------------
|               DescribeAlarms                 |
+-----------------------------------------------+
|  arn:aws:automate:us-east-1:ec2:recover       |
|  arn:aws:sns:us-east-1:216772830502:avisos-pln  |
+-----------------------------------------------+
```

Figura 34. Acciones finales de mon-pln-sistema: recover y el tema SNS

Para pensar: si la recuperación automática simplificada ya está activada, ¿qué aporta esta alarma?

Control y visibilidad. La recuperación simplificada actúa sola y en silencio; con la alarma decido cuándo se dispara (2 minutos seguidos), queda historial en CloudWatch, aparece en el panel y, sobre todo, alguien recibe un correo cuando la instancia se ha movido de host. Además la alarma se puede combinar con otras acciones o notificaciones.

Parte 5: Alarma de CPU

He creado mon-pln-cpu con el asistente de CloudWatch: CPUUtilization de mon-pln, Promedio, 1 minuto, Estático mayor que 70, 3 de 5 puntos de datos, datos ausentes tratados como ausentes y notificación a avisos-pln tanto al entrar en alarma como al volver a OK.

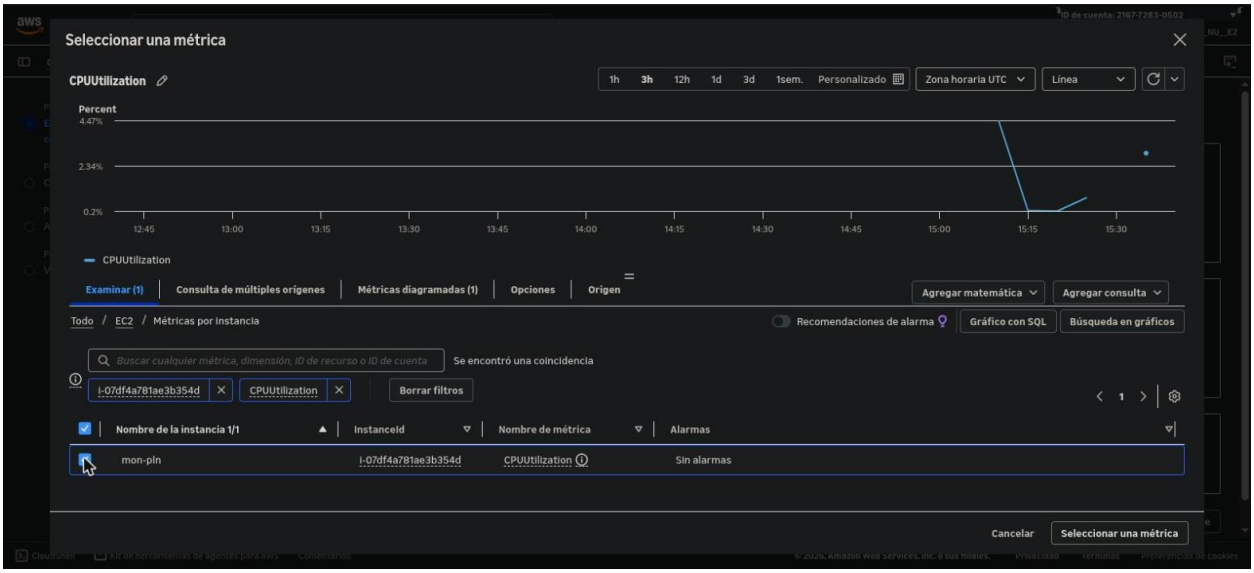


Figura 35. Selección de la métrica CPUUtilization de mon-pln

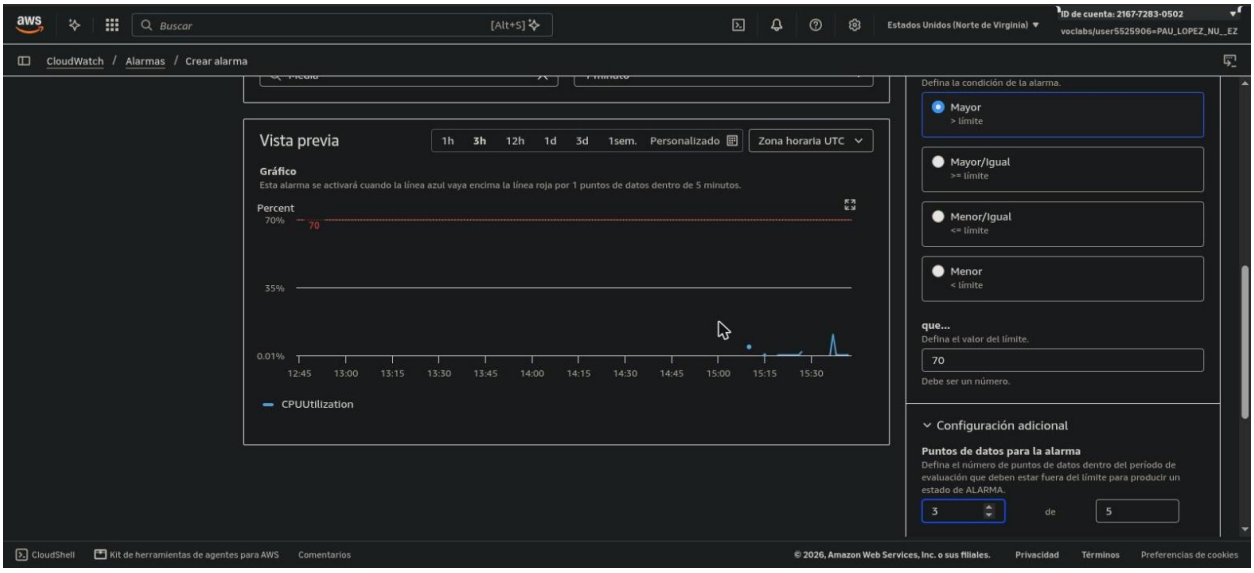


Figura 36. Condición mayor que 70: la línea roja marca el umbral

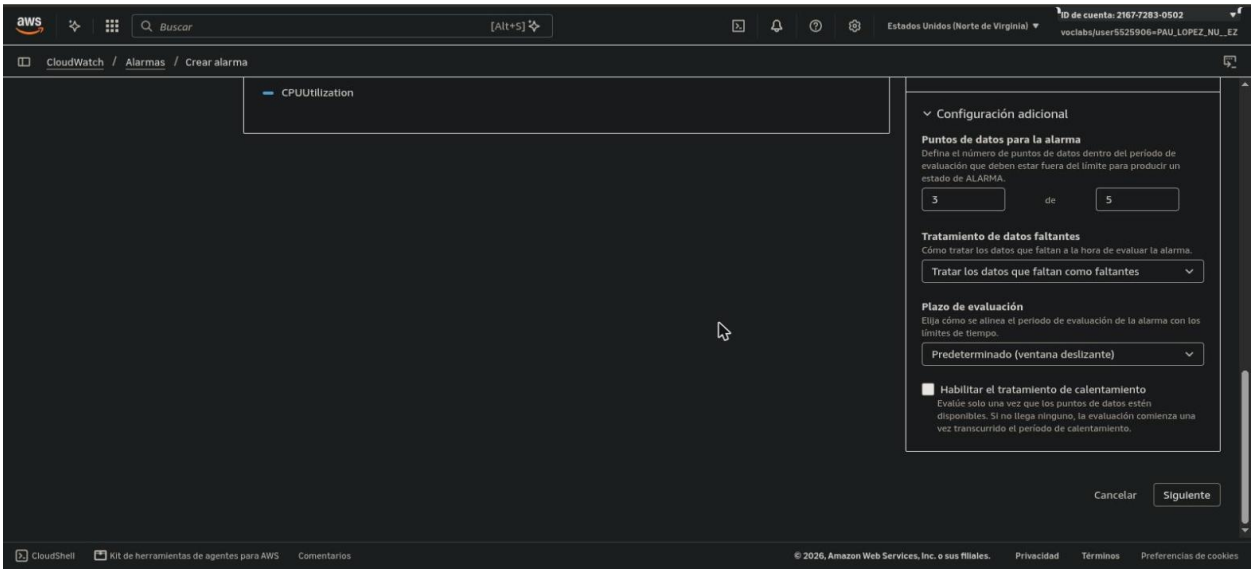


Figura 37. 3 de 5 puntos de datos y datos ausentes como ausentes

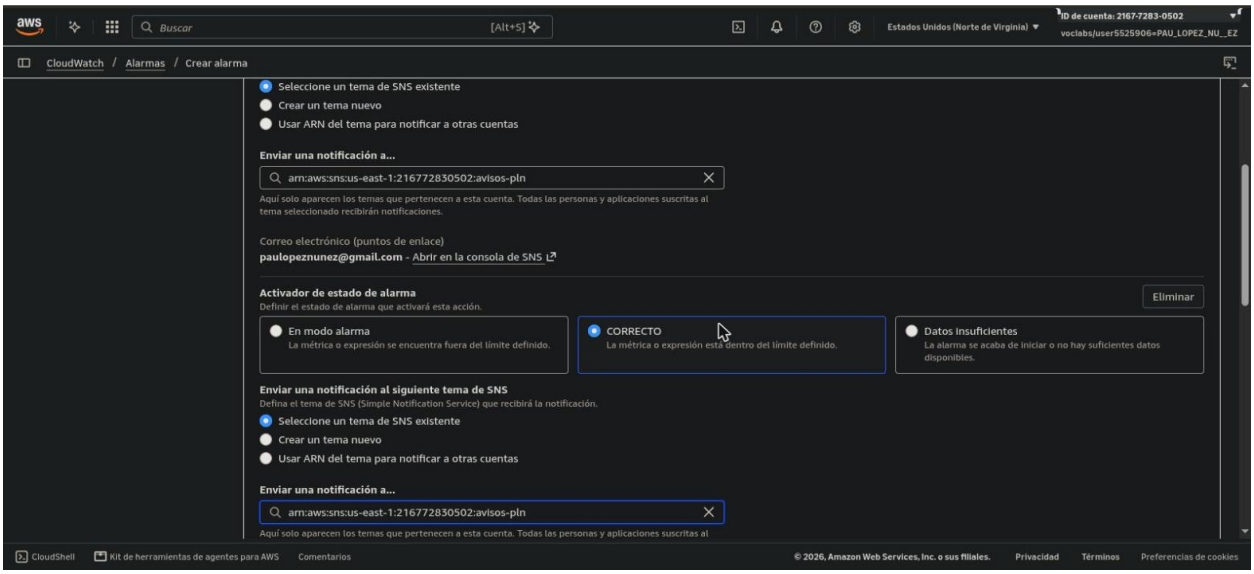


Figura 38. Notificaciones en alarma y en OK

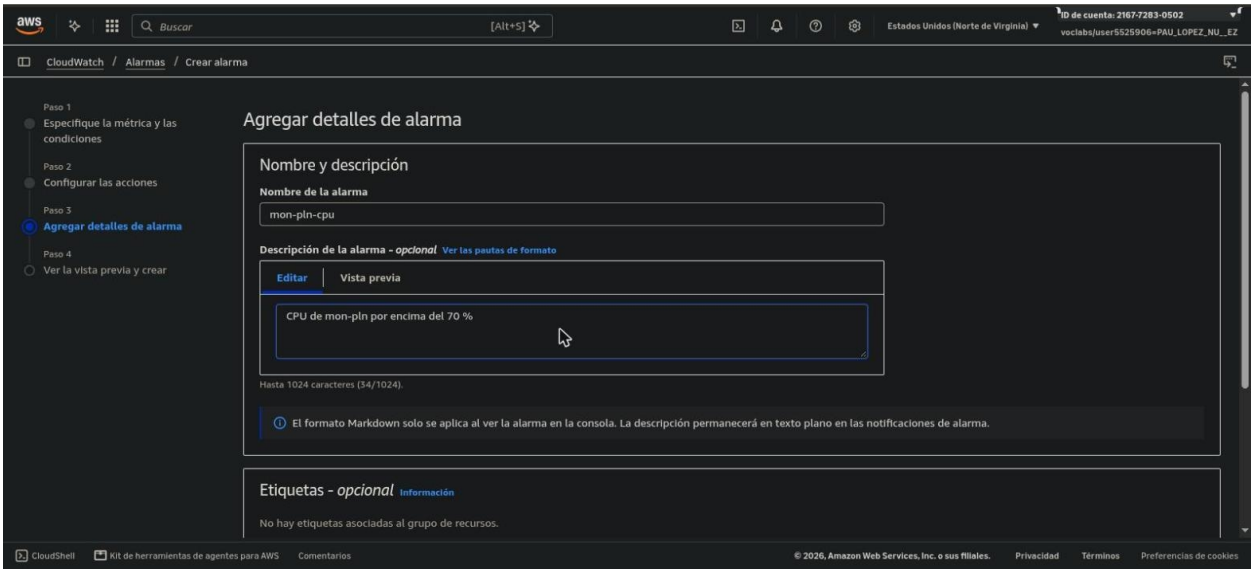


Figura 39. Nombre y descripción

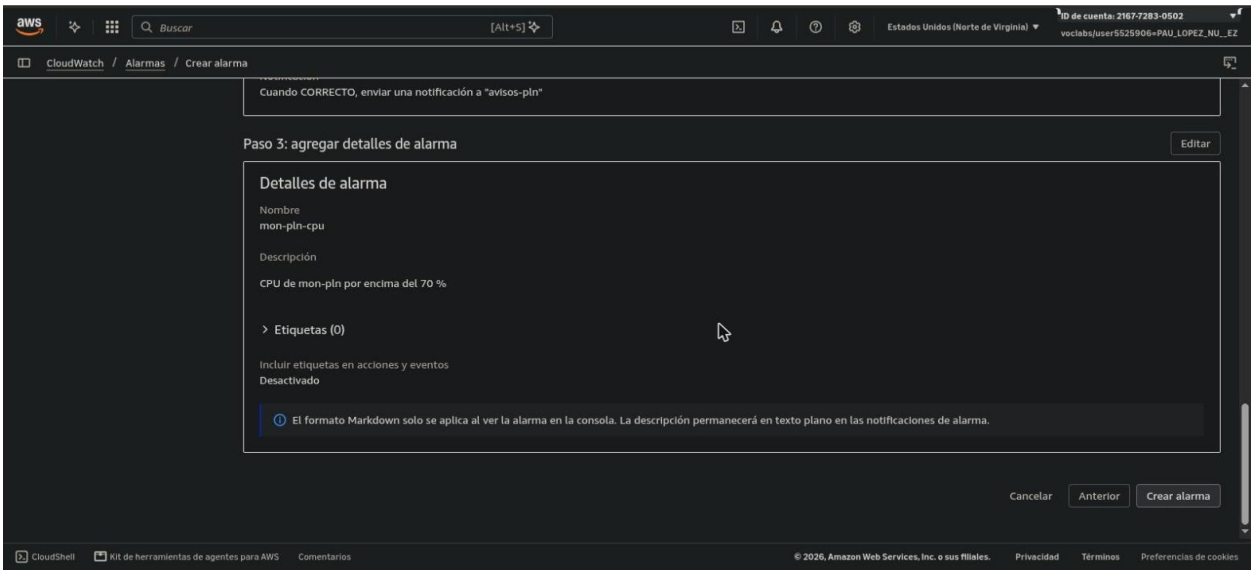


Figura 40. Vista previa

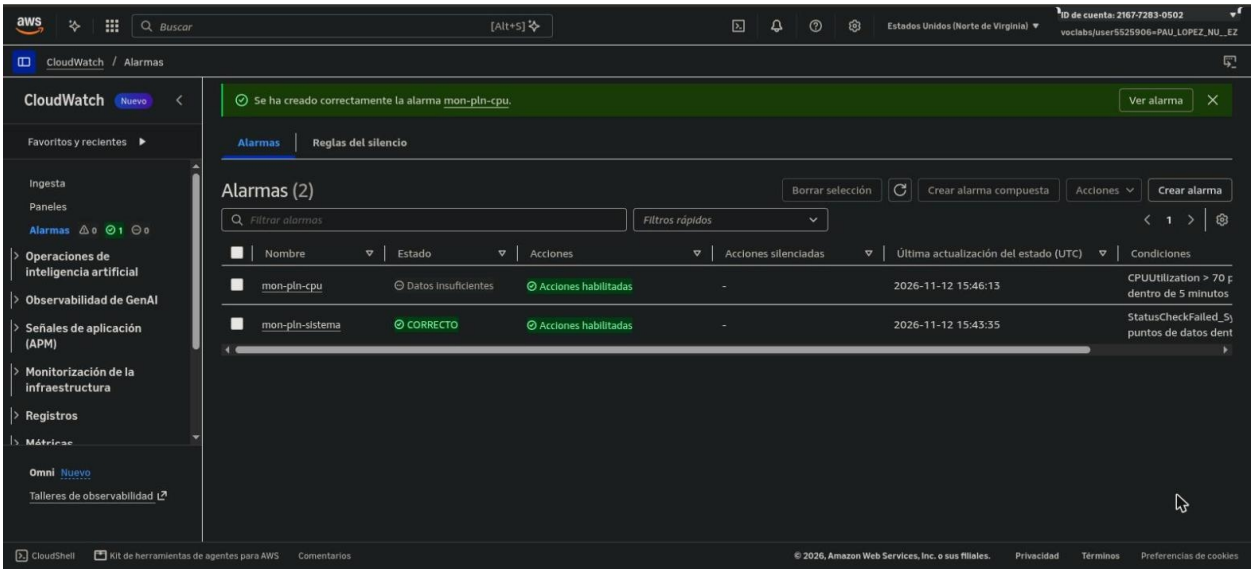


Figura 41. Alarma creada

Después me he conectado por SSH (usuario ubuntu) y he lanzado stress-ng sobre todas las CPU durante 8 minutos:

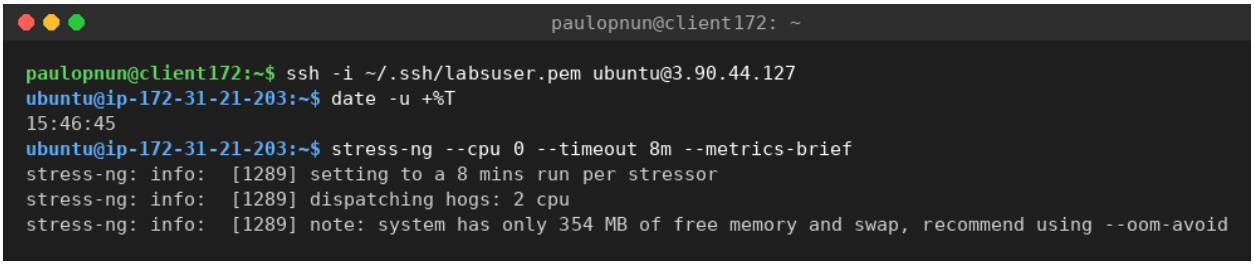


Figura 42. stress-ng --cpu 0 --timeout 8m --metrics-brief

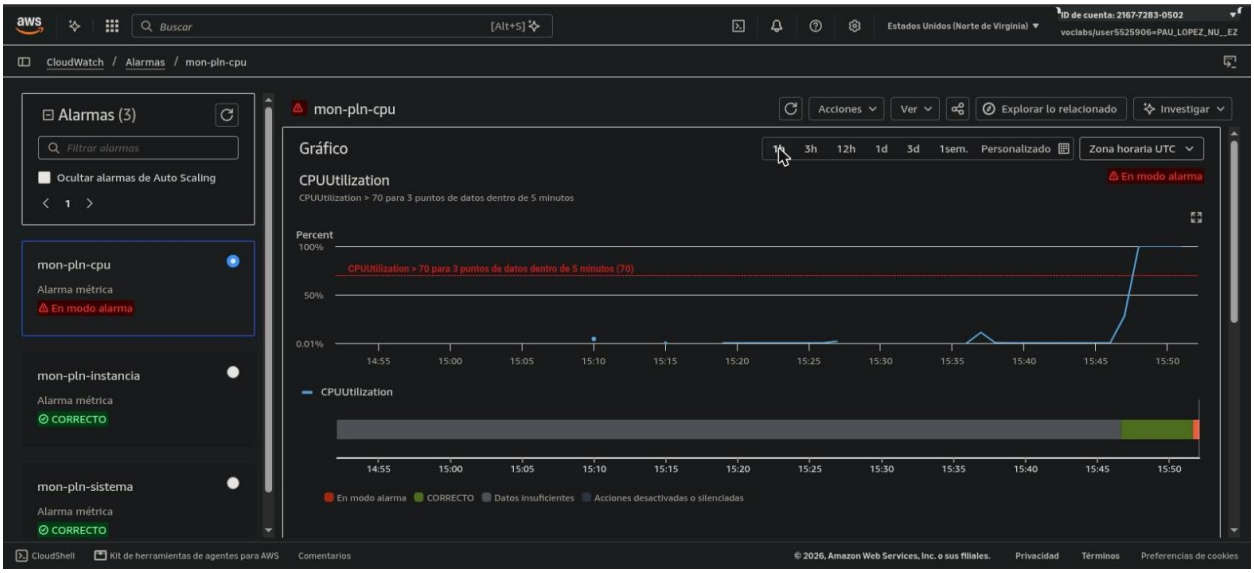


Figura 43. La CPU sube al 100 % y la alarma pasa a En alarma

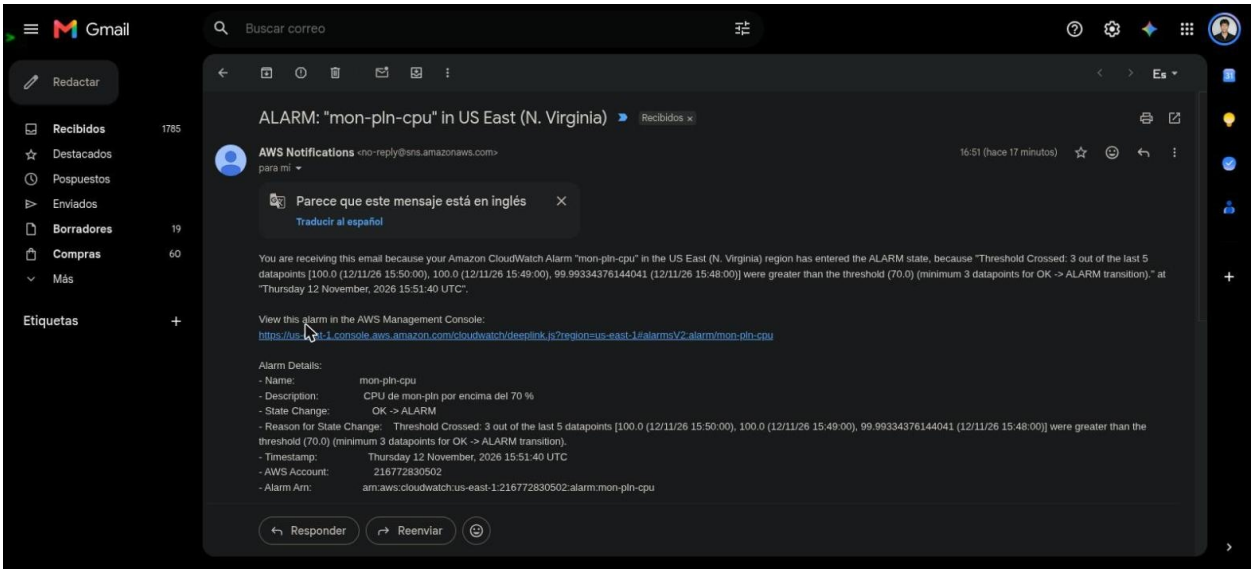


Figura 44. Correo de ALARM con los tres datos que superaron el umbral

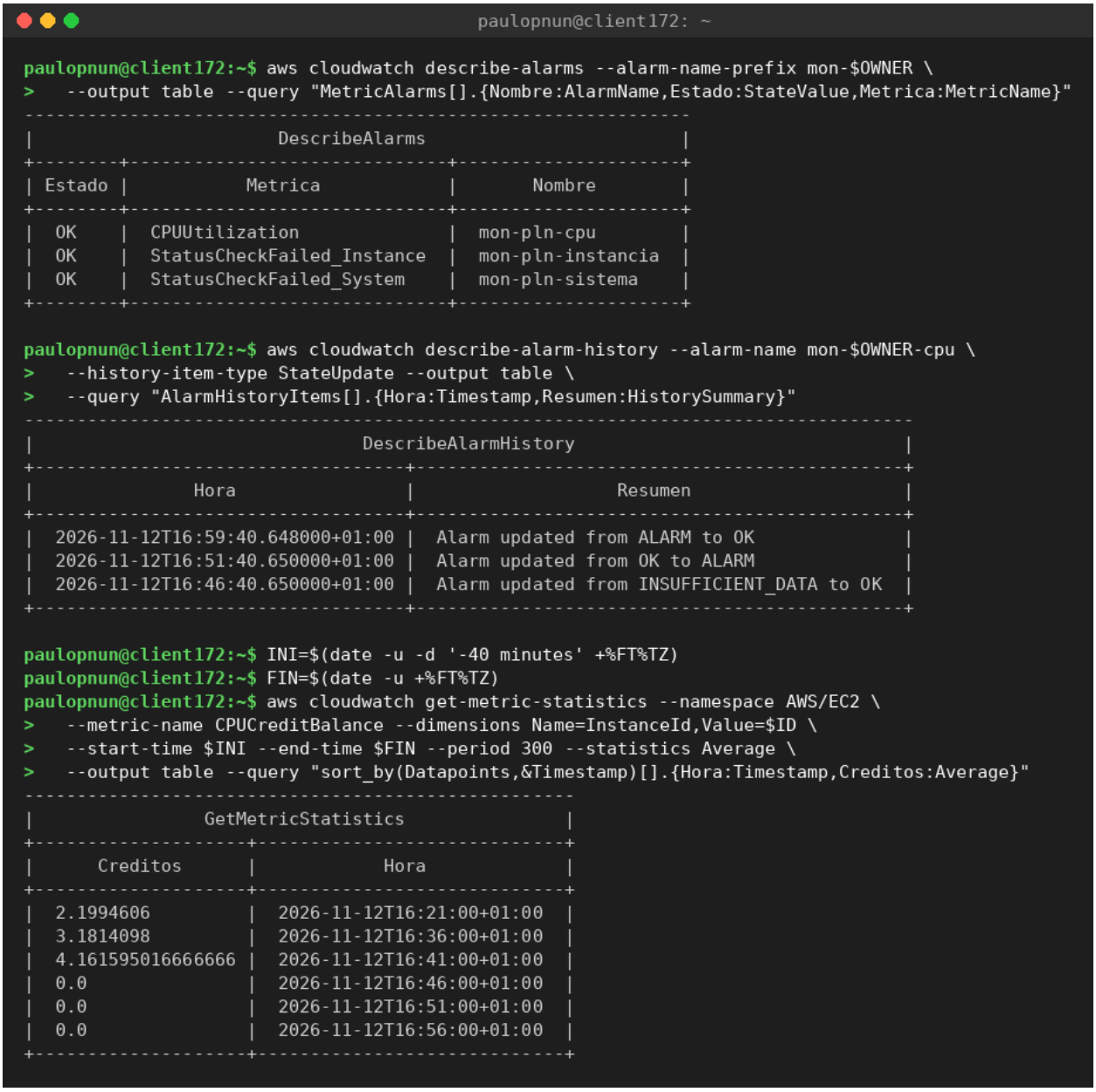


Figura 45. Estado de las alarmas, historial de mon-pln-cpu y CPUCreditBalance

Respuestas a los ejercicios:

- Historial: Datos insuficientes a OK a las 16:46:40, OK a En alarma a las 16:51:40 y En alarma a OK a las 16:59:40, poco después de que terminara stress-ng.
- Desde que lancé stress-ng (15:46:45 UTC, 16:46:45 hora local) hasta el cambio a En alarma pasaron unos 5 minutos. El primer minuto la CPU aún no estaba alta del todo; hacían falta 3 datos de 1 minuto por encima de 70 (15:48, 15:49 y 15:50, como dice el correo), y cada dato llega a CloudWatch con 1 o 2 minutos de retraso y la alarma se evalúa cada minuto.
- CPUCreditBalance: antes de la prueba la instancia iba acumulando créditos (2,2 → 3,2 → 4,2) y durante la prueba bajó a 0. Las t3 están en modo ilimitado, así que no se ralentizó, pero el exceso sobre la línea base se cobra; por eso hay que lanzar stress-ng siempre con --timeout.

Para pensar: con 3 de 5, un pico de dos minutos no dispara la alarma. ¿Qué pasaría con 1 de 1? ¿Y con 5 de 5? ¿Qué elegirías para una alarma que despierta a alguien a las tres de la madrugada?

Con 1 de 1 cualquier pico de un minuto (una actualización, un cron) la dispararía: muchas falsas alarmas. Con 5 de 5 solo salta si la CPU está alta los cinco minutos seguidos, y un solo minuto por debajo la reinicia, así que es lenta y se puede escapar un problema real que oscila. Para despertar a alguien elegiría algo tolerante a picos pero que no dependa de que todos los datos sean malos, por ejemplo 3 de 5 o 4 de 5 con periodos de 1 minuto, o 3 de 3 con periodos de 5 minutos.

Para pensar: ¿qué tratamiento de datos ausentes elegirías para una alarma que vigila que un proceso nocturno envía una métrica cada 5 minutos?

breaching (tratar los datos ausentes como incumplimiento). Si el proceso muere, deja de llegar la métrica, y eso es justo el fallo que quiero detectar. Con missing la alarma acabaría en Datos insuficientes sin avisar, con ignore se quedaría en el estado anterior (OK) y con notBreaching contaría como que todo va bien.

Parte 6: Reinicio ante un fallo de la instancia

He creado mon-pln-instancia desde EC2: StatusCheckFailed_Instance, Máximo, 1 minuto, mayor o igual que 1 en 3 de 3, acción Reiniciar y notificación a avisos-pln.

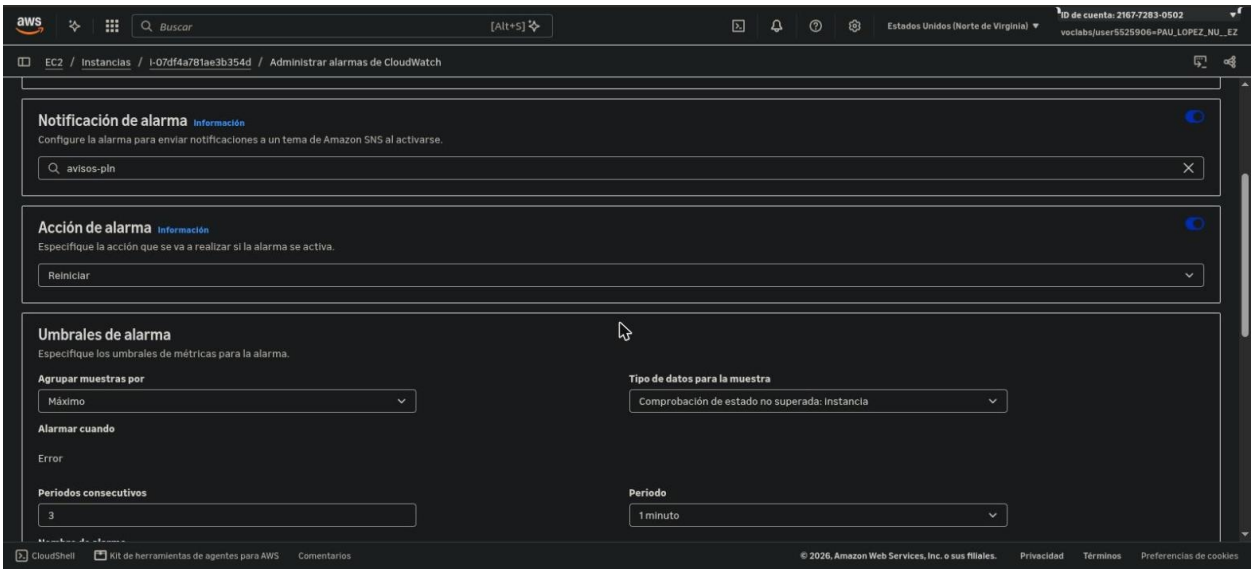


Figura 46. Configuración de mon-pln-instancia

Con la alarma en OK me he conectado por SSH y he desactivado la interfaz con sudo ip link set ens5 down a las 17:09. La sesión SSH se quedó colgada y a partir de ahí la instancia no respondía por SSH (Connection timed out), mientras que la consola seguía mostrando el estado del sistema correcto.

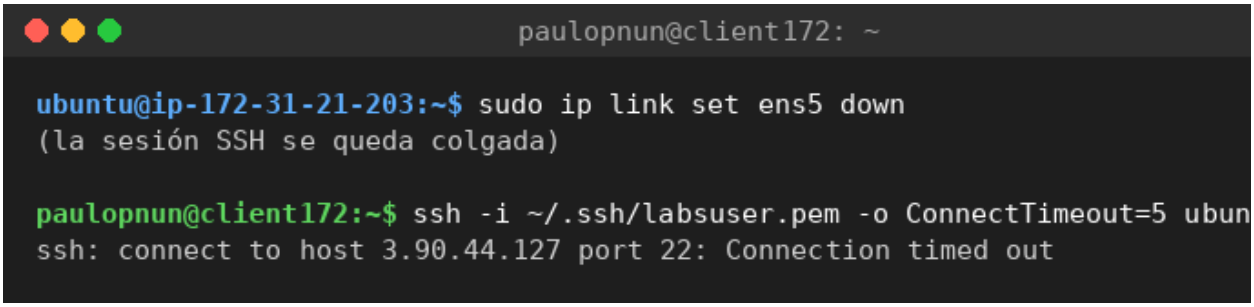


Figura 47. Caída de la red de la instancia

Por falta de tiempo de laboratorio no llegué a esperar el cambio de la alarma a En alarma ni el reinicio: la comprobación de la instancia tarda varios minutos en marcar el fallo y la alarma necesita después 3 minutos seguidos. Lo esperado es que, unos 3 a 5 minutos después, la comprobación de accesibilidad de la instancia falle, la alarma pase a En alarma, ejecute arn:aws:automate:us-east-1:ec2:reboot y avise por correo; como el cambio con ip link no es permanente, tras el reinicio la interfaz vuelve a estar activa.

Para pensar: si el fallo fuera permanente, ¿se reiniciaría la instancia una y otra vez?

No. Las acciones se ejecutan cuando la alarma cambia de estado, no mientras sigue en alarma. Si después del reinicio la comprobación sigue fallando, la alarma se queda En alarma y no vuelve a ejecutar la acción; solo se repetiría si pasara a OK y luego volviera a fallar. Habría que entrar a arreglarlo (por ejemplo con la consola serie o separando el volumen).

Para pensar: ¿por qué no tiene sentido usar Recuperar para este fallo, ni Reiniciar para un fallo del sistema?

Recuperar mueve la instancia a otro host, pero el problema está dentro del sistema operativo y viajaría con ella: el disco y la configuración son los mismos. Reiniciar un fallo del sistema no sirve porque el reinicio se hace en el mismo host físico que está fallando; hay que cambiar de host, que es lo que hace Recuperar.

Parte 7: Comprobación del estado de la aplicación

La comprobación se hace desde la red de AWS dentro de la VPC, así que he añadido a mon-pln-sg una regla HTTP desde 172.31.0.0/16:

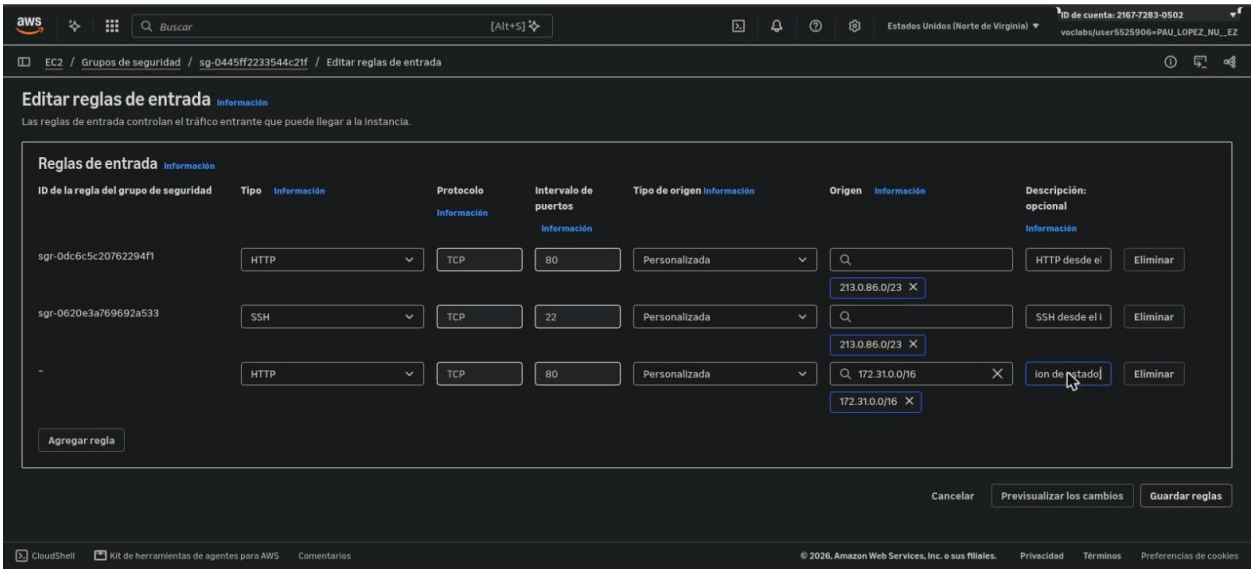


Figura 50. Regla HTTP desde la VPC

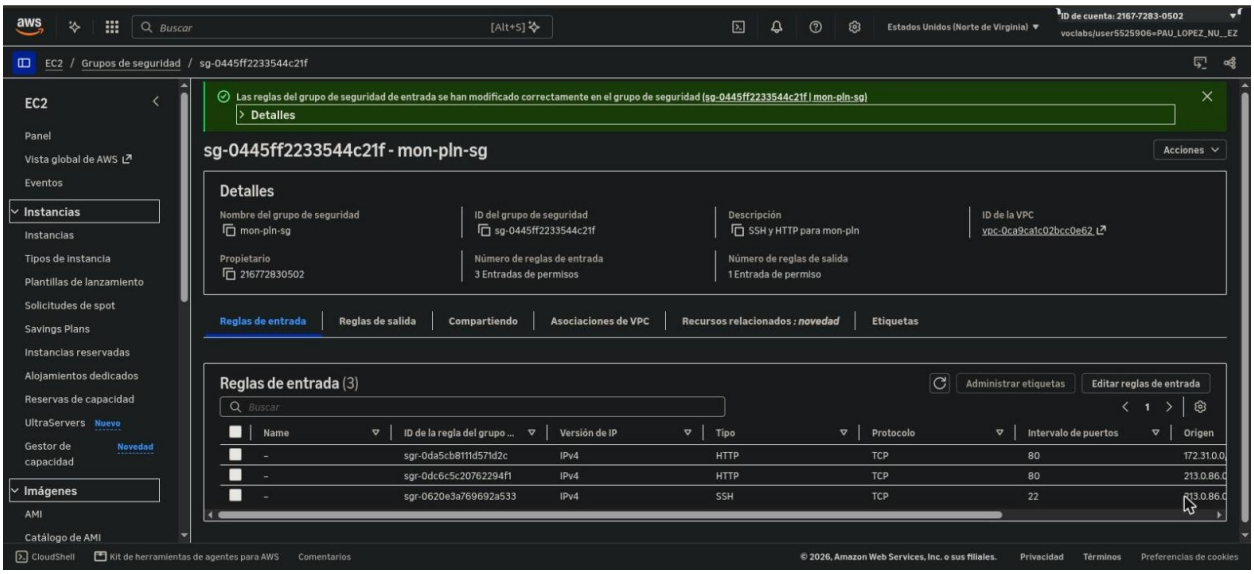


Figura 51. Reglas de entrada de mon-pln-sg

He creado la comprobación apache-pln (HTTP, puerto 80, ruta /, 5 s de espera, código 200, umbrales 2 y 2, agregación Incluido, rutas de red predeterminadas) y la he asociado a la instancia por ID. La consola también pedía el periodo de gracia de inicialización, que el enunciado no menciona; le he puesto 60 segundos.

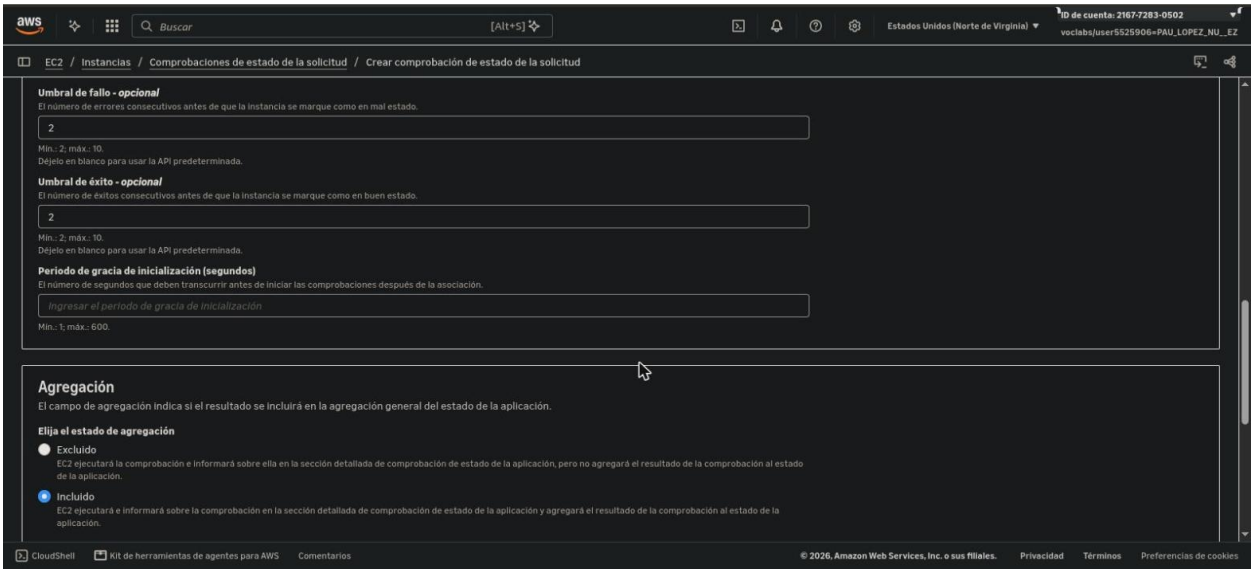


Figura 52. Controles y umbrales de apache-pln

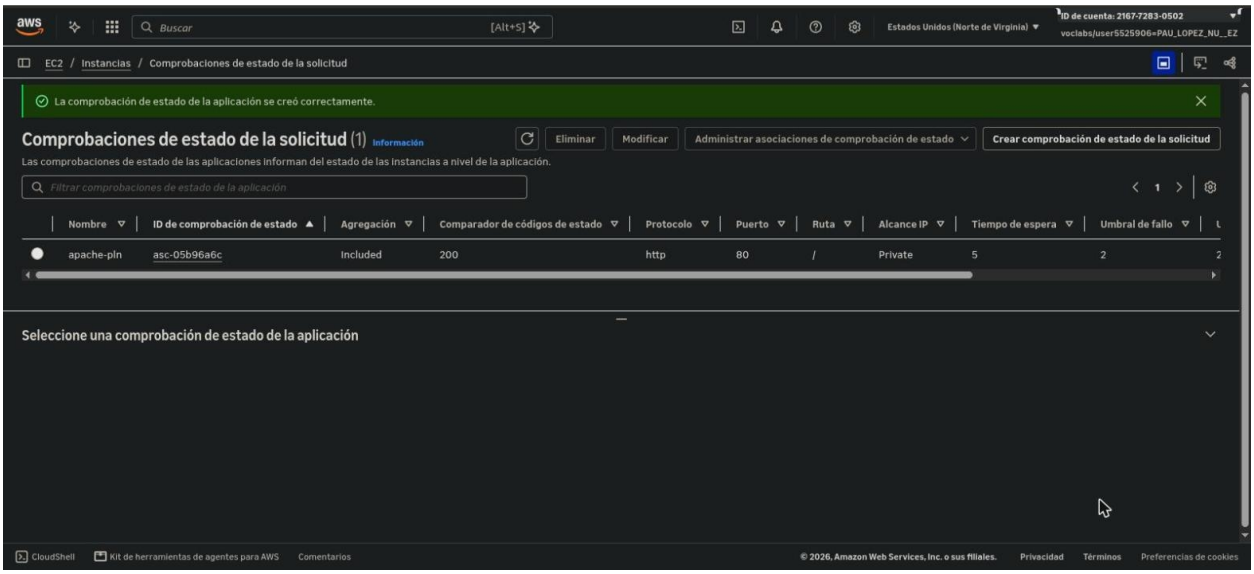


Figura 53. Comprobación apache-pln (asc-05b96a6c)

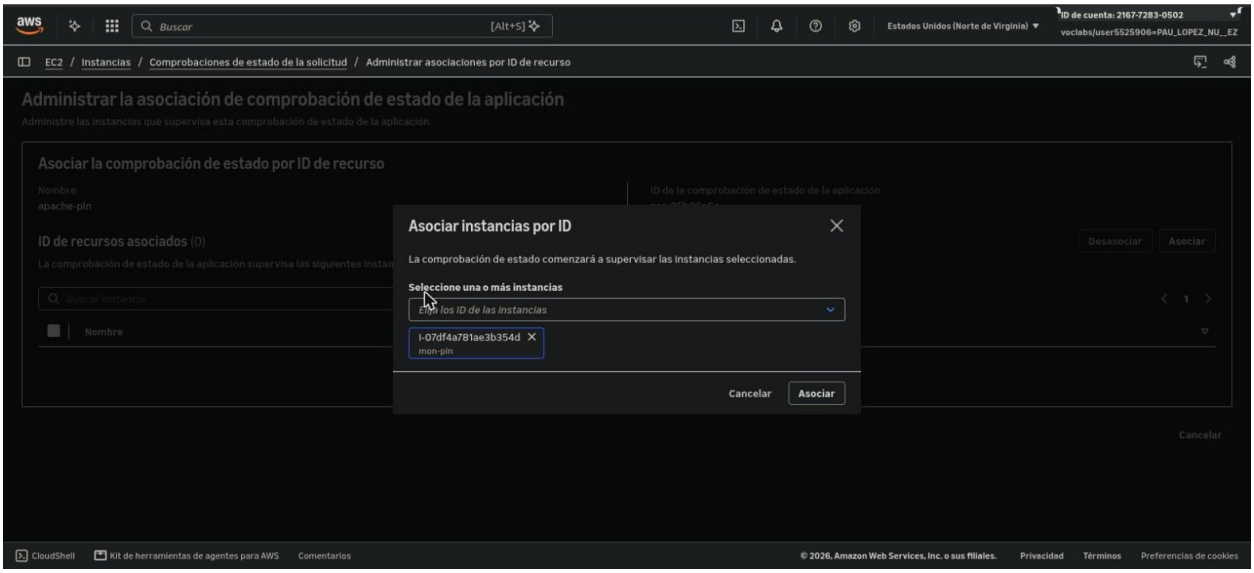


Figura 54. Asociación por ID de instancia

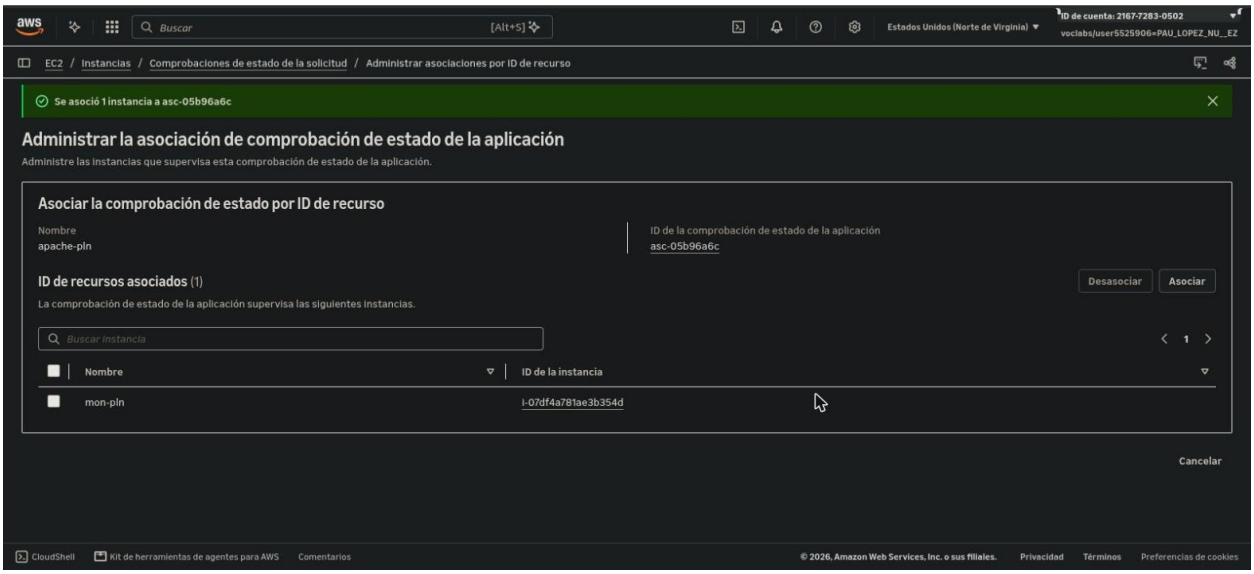


Figura 55. Instancia asociada

Al parar Apache con sudo systemctl stop apache2 la comprobación pasa a Error con el motivo ConnectionRefused, mientras que las del sistema, la instancia y EBS siguen superadas. Al arrancarlo de nuevo vuelve a Aprobado.

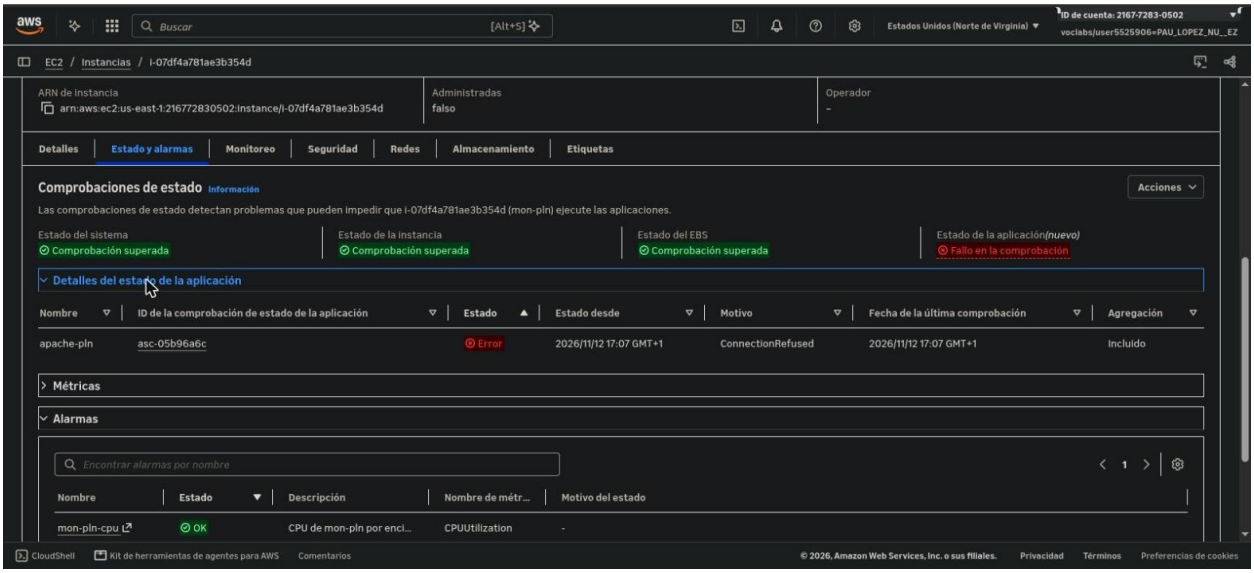


Figura 56. Apache parado: Error con ConnectionRefused y el resto de comprobaciones superadas

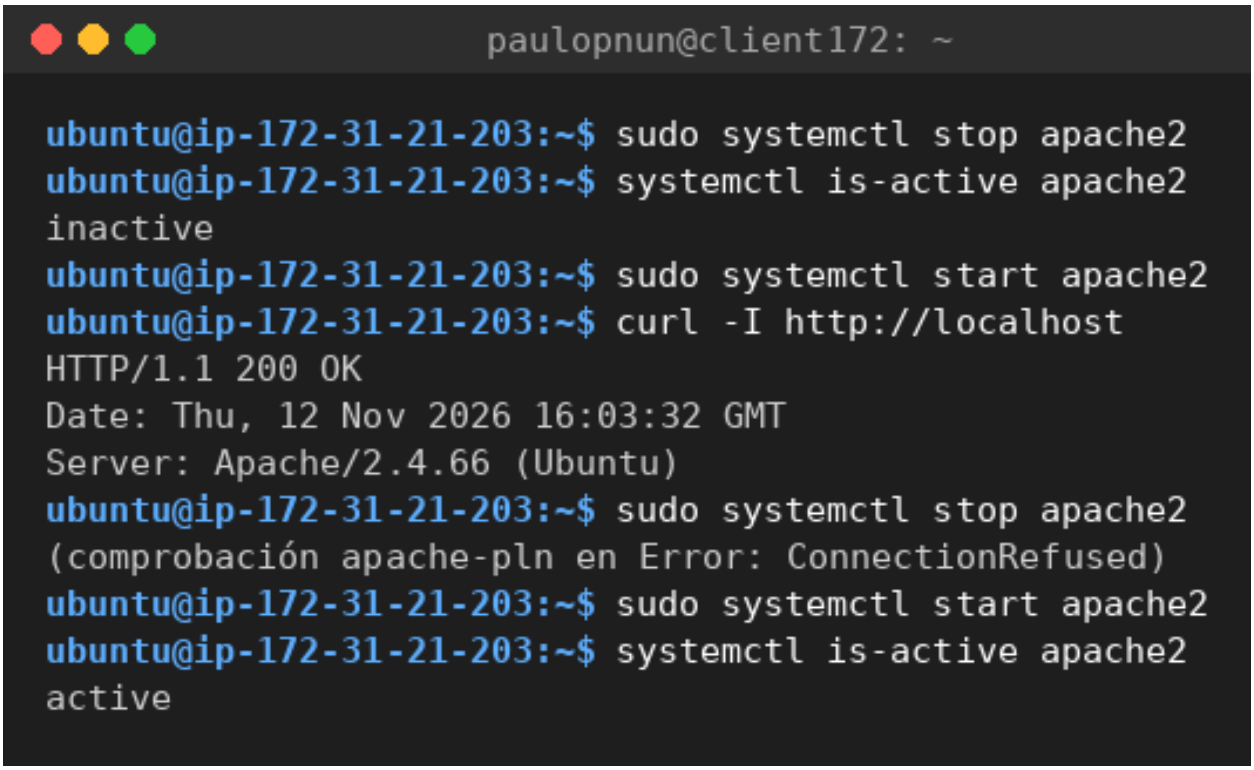


Figura 57. Parada y arranque de Apache

Nota: la primera vez paré Apache antes de que la comprobación hubiera llegado a Aprobado y el motivo fue ConnectionTimeout (figura 58). Al arrancar Apache pasó a Aprobado, así que la regla de la VPC era correcta; al repetir la parada ya con la comprobación en Aprobado salió ConnectionRefused, que es lo esperado.

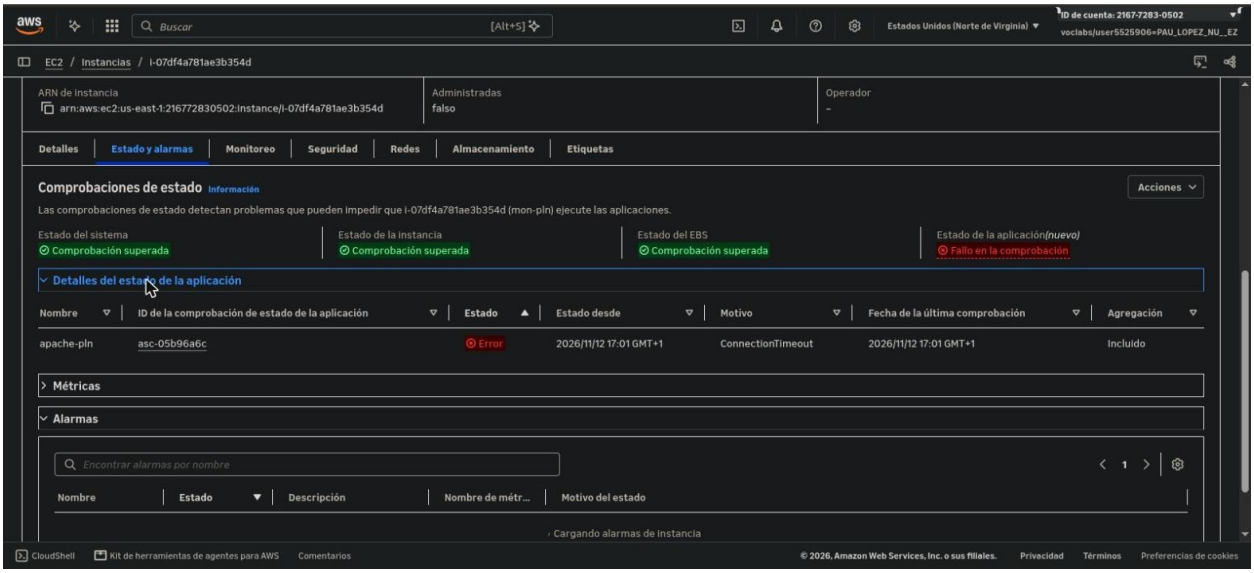


Figura 58. Primer intento: ConnectionTimeout

Para pensar: con Apache parado, ¿qué dicen las comprobaciones del sistema y de la instancia? ¿Por qué no basta con ellas?

Las dos siguen superadas, porque el host funciona y el sistema operativo responde por la red. Solo miran si la máquina está viva, no si el servicio que corre en ella responde; por eso una web caída pasa desapercibida si no hay una comprobación de la aplicación.

Para pensar: ¿qué diferencia hay entre ConnectionRefused y ConnectionTimeout?

ConnectionRefused significa que el paquete llegó a la instancia y el sistema contestó que no hay nada escuchando en ese puerto: la red está bien y el servicio está parado. ConnectionTimeout significa que no llegó ninguna respuesta: suele ser un problema de red, como un grupo de seguridad que no permite el tráfico, una ruta o una instancia sin red.

Parte 8: Panel de CloudWatch

He creado panel-pln con un widget de línea de CPUUtilization con periodo de 1 minuto, un widget de estado de alarma con las tres alarmas y un widget de número con el máximo de StatusCheckFailed.

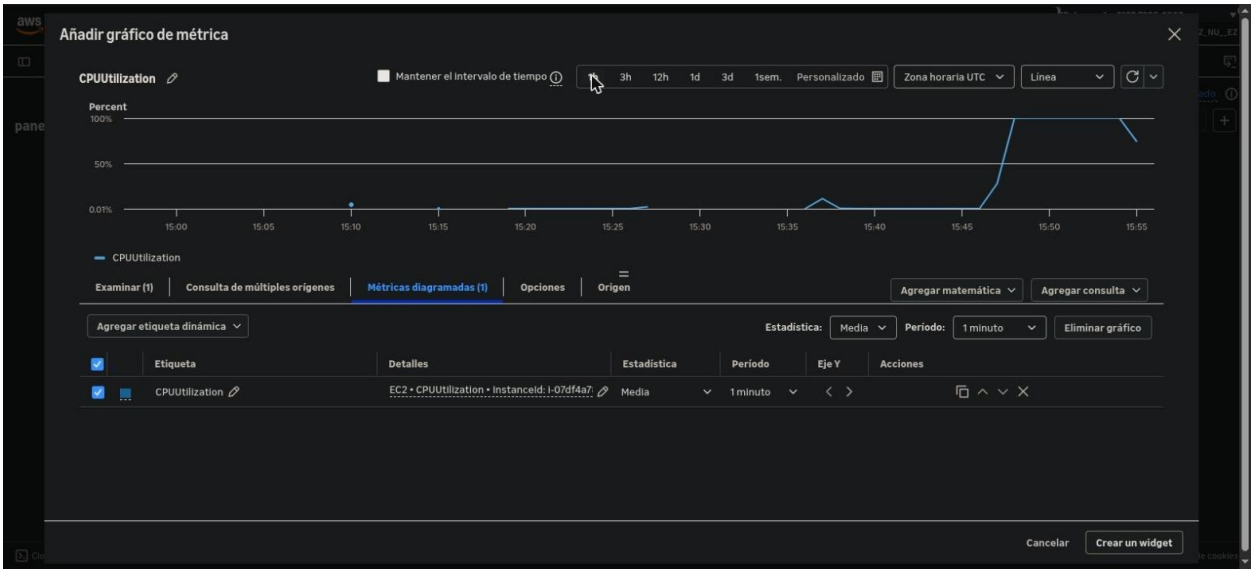


Figura 59. Widget de línea con CPUUtilization a 1 minuto

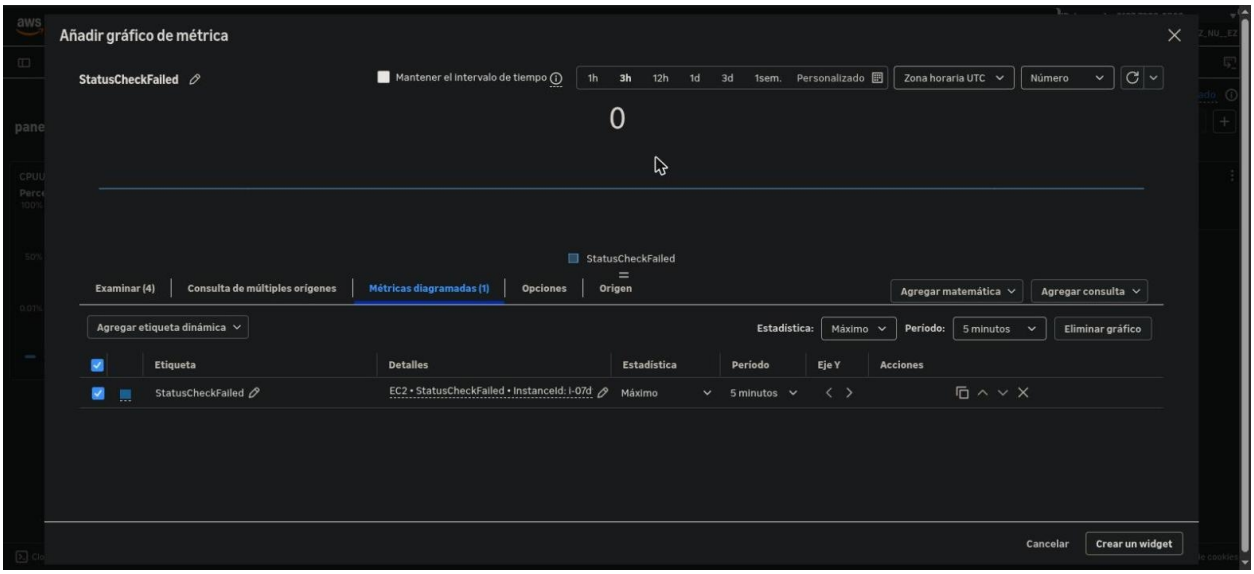


Figura 60. Widget de número con el máximo de StatusCheckFailed

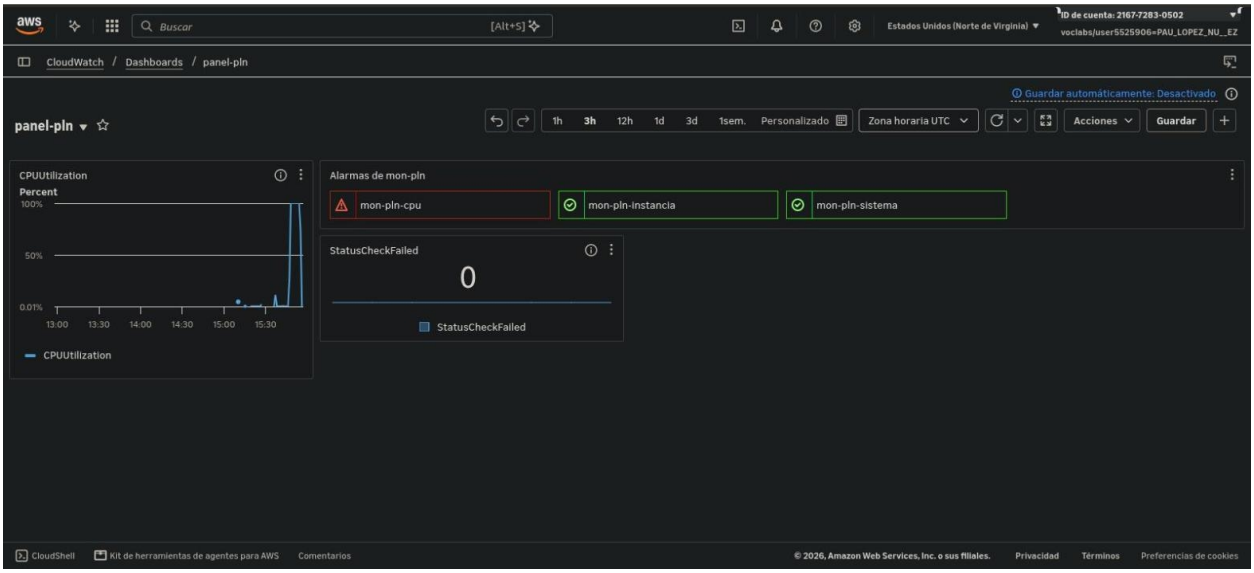


Figura 61. Panel panel-pln con la prueba de carga y el estado de las alarmas

El panel es un documento JSON. Lo he guardado en evidencias/panel-pln.json y lo he vuelto a aplicar con put-dashboard, que es lo que permitiría crear el mismo panel en otra cuenta o región:

```
paulopnun@client172: ~
paulopnun@client172:~$ aws cloudwatch put-dashboard --dashboard-nam
>   --dashboard-body file://$EVID/panel-pln.json
{
    "DashboardValidationMessages": []
}

paulopnun@client172:~$ aws cloudwatch list-dashboards --output tabl
>   --query "DashboardEntries[].{Panel:DashboardName,Bytes:Size}"
-----------------------
|    ListDashboards    |
+--------+-------------+
|  Bytes |    Panel    |
+--------+-------------+
|  834   |  panel-pln  |
+--------+-------------+
```

Figura 62. put-dashboard con el JSON del panel

Para pensar: los tres primeros paneles de la cuenta son gratuitos. ¿Por qué conviene tener pocos paneles bien pensados?

Porque a partir del tercero se pagan, y sobre todo porque un operador necesita ver en una pantalla lo importante. Con un panel por recurso hay que ir saltando de uno a otro y es fácil no ver una alarma; es mejor agrupar por servicio o aplicación y poner solo las métricas y alarmas que llevan a tomar una decisión.

Ampliación: alarmas desde la CLI

He creado mon-pln-cpu-cli con put-metric-alarm completando los huecos: --metric-name CPUUtilization, --statistic Average, --comparison-operator GreaterThanThreshold y --datapoints-to-alarm 3. Al compararla con describe-alarms coincide con mon-pln-cpu en todo salvo el nombre.

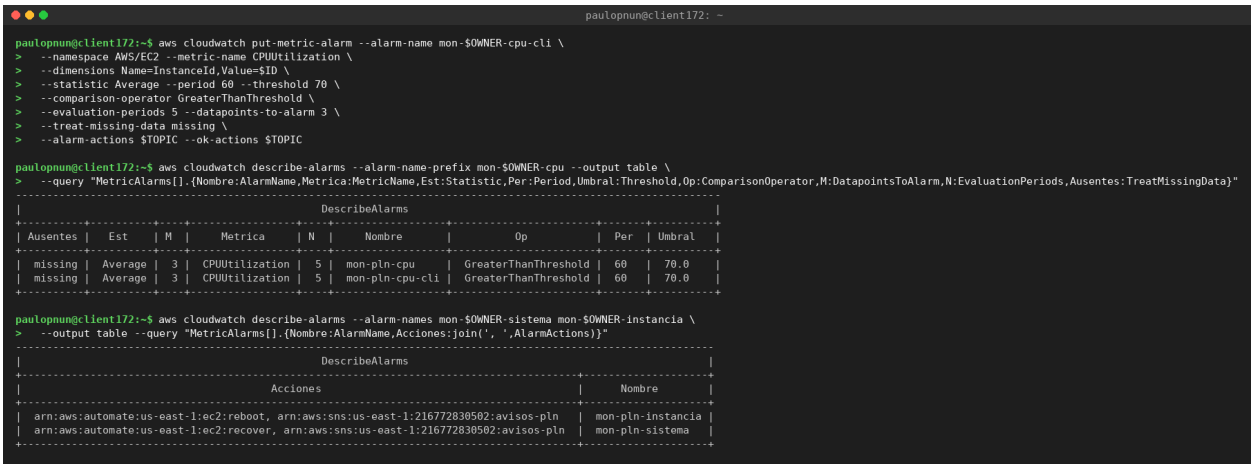


Figura 63. put-metric-alarm, comparación con mon-pln-cpu y ARN de las acciones de EC2

La acción de recuperar es arn:aws:automate:us-east-1:ec2:recover, como se ve en mon-pln-sistema (y la de reiniciar, arn:aws:automate:us-east-1:ec2:reboot, en mon-pln-instancia).

Parte 9: Fin de la práctica

- He eliminado la comprobación apache-pln con delete-application-status-check, porque cuesta unos 0,01 USD por hora.
- He borrado la alarma duplicada mon-pln-cpu-cli con delete-alarms. La alarma opcional mon-pln-aplicacion no la creé.
- He detenido mon-pln. Se conservan la instancia, mon-pln-sg, el tema avisos-pln con la suscripción confirmada, panel-pln y las tres alarmas para la siguiente actividad.

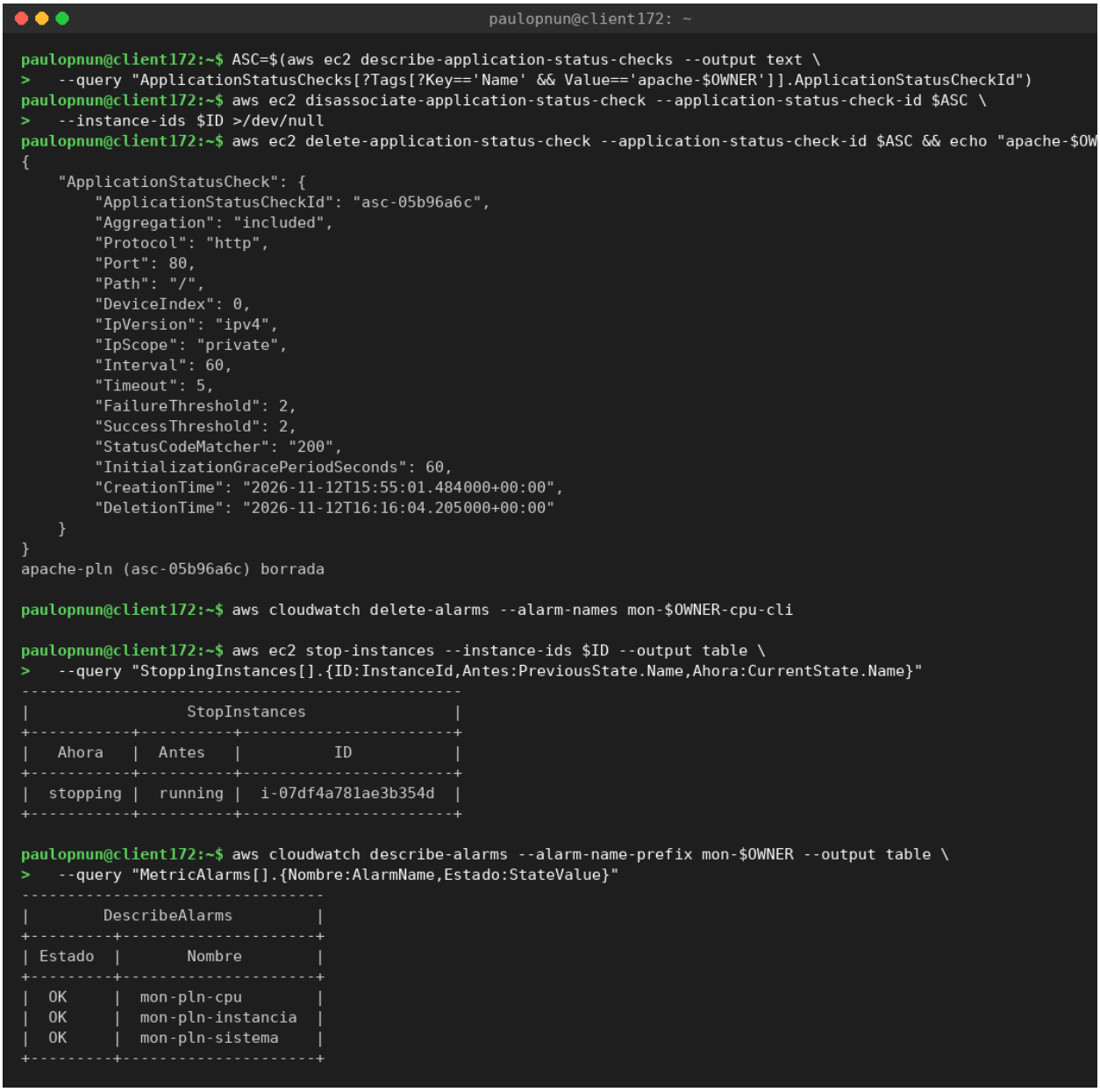


Figura 50. Limpieza final

Conclusión

Lo que más me ha quedado claro es que cada comprobación detecta un tipo de fallo distinto y pide una acción distinta: la del sistema, recuperar en otro host; la de la instancia, reiniciar; y la de la aplicación, avisar, porque ninguna de las otras dos se entera de que Apache está caído. También he visto en la práctica cómo influyen el periodo, la regla M de N y el retraso de los datos en lo que tarda una alarma en saltar, y que una alarma que actúa sin avisar a nadie se queda a medias, por eso todas las alarmas acaban notificando a avisos-pln.